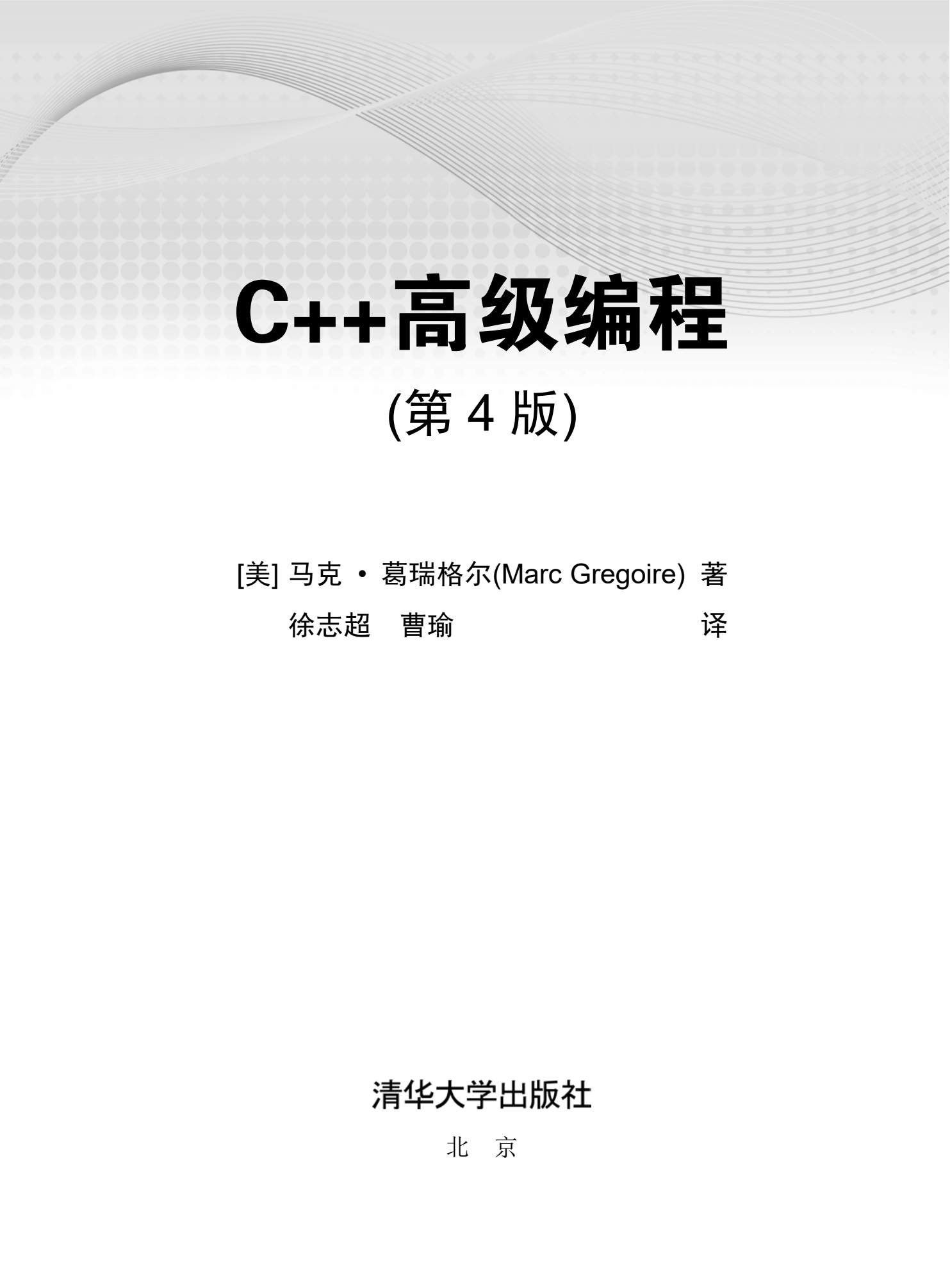




C++高级编程

(第4版)

[美] 马克·葛瑞格尔(Marc Gregoire) 著

徐志超 曹瑜

译

清华大学出版社

北 京

Professional C++, 4th Edition

Marc Gregoire

EISBN: 978-1-119-42130-6

Copyright © 2018 by John Wiley & Sons, Inc., Indianapolis, Indiana

All Rights Reserved. This translation published under license.

Trademarks: Wiley, the Wiley logo, Wrox, the Wrox logo, Programmer to Programmer, and related trade dress are trademarks or registered trademarks of John Wiley & Sons, Inc. and/or its affiliates, in the United States and other countries, and may not be used without written permission. Visual Studio is a registered trademark of Microsoft Corporation. All other trademarks are the property of their respective owners. John Wiley & Sons, Inc., is not associated with any product or vendor mentioned in this book.

本书中文简体字版由 Wiley Publishing, Inc. 授权清华大学出版社出版。未经出版者书面许可, 不得以任何方式复制或抄袭本书内容。

北京市版权局著作权合同登记号 图字: 01-2018-4099

本书封面贴有 Wiley 公司防伪标签, 无标签者不得销售。

版权所有, 侵权必究。侵权举报电话: 010-62782989 13701121933

图书在版编目(CIP)数据

C++高级编程: 第4版 / (美)马克·葛瑞格尔(Marc Gregoire) 著; 徐志超, 曹瑜 译. —北京: 清华大学出版社, 2019

书名原文: Professional C++, 4th Edition

ISBN 978-7-302-52631-5

I. ①C… II. ①马… ②徐… ③曹… III. ①C++语言—程序设计 IV. ①TP312.8

中国版本图书馆 CIP 数据核字(2019)第 046888 号

责任编辑: 王 军

装帧设计: 孔祥峰

责任校对: 成凤进

责任印制:

出版发行: 清华大学出版社

网 址: <http://www.tup.com.cn>, <http://www.wqbook.com>

地 址: 北京清华大学学研大厦 A 座 邮 编: 100084

社 总 机: 010-62770175

邮 购: 010-62786544

投稿与读者服务: 010-62776969, c-service@tup.tsinghua.edu.cn

质 量 反 馈: 010-62772015, zhiliang@tup.tsinghua.edu.cn

印 刷 者:

装 订 者:

经 销: 全国新华书店

开 本: 190mm×260mm 印 张: 46.75 字 数: 1580 千字

版 次: 2019 年 4 月第 1 版 印 次: 2019 年 4 月第 1 次印刷

定 价: 138.00 元

产品编号:

译者序

在中国大陆的程序员圈子中，常将 C++ 读作“C 加加”；而西方国家的程序员常将 C++ 读作“C plus plus”或“CPP”。C++ 最初作为 C 语言的增强版出现，此后不断融入新特性，比如虚函数、运算符重载、多重继承、模板、异常、RTTI、结构化绑定、嵌套的名称空间等。

多年来，C++ 都是编写性能卓越、功能强大的企业级面向对象程序的事实标准语言。尽管 C++ 语言已经风靡全球，但这门语言却极难完全掌握，即使是经验丰富的 C++ 程序员也未必了解 C++ 中某些很有用的特性。

要成为一名专业的 C++ 程序员，必须扎实理解 C++ 语言的工作原理，了解不同编程方法论和软件开发过程——从设计和编码，到测试、调试等，以便更好地和团队协作。要知道良好设计的重要性，领略面向对象编程的理论，掌握卓越的调试技能，探索可重用的库和常用的设计模式，了解可重用的思想，以大幅提升日常工作效率。

这本讲解 C++17 的著作将帮助读者全面透彻地掌握 C++ 语言的功能，包罗 C++ 语言的一切，分享真实范例，展现 C++17 的新工具和功能，详述如何在真实世界中使用 C++，揭示最新版 C++ 带来的显著变化，解密 C++ 中鲜为人知的特性，探索编程方法论、可重用的设计模式和良好的编程风格，阐述如何设计可充分利用 C++ 语言功能的高效解决方案。本书深入探讨 C++ 语言功能集的更复杂元素，并讲解避开常见陷阱的技巧。本书提供详尽的编程指南，紧贴实际，是编程人员深入挖掘 C++ 的理想工具。

本书包括 5 部分。第 I 部分是 C++ 基础速成教程，第 II 部分介绍 C++ 设计方法论，第 III 部分从专业角度分析 C++ 编程技术，第 IV 部分讲解如何真正掌握 C++ 的高级功能，第 V 部分重点介绍 C++ 软件工程技术。最后的附录 A 列出了在 C++ 技术面试中取得成功的指南，附录 B 列出带注解的参考文献，附录 C 总结标准的 C++ 头文件，附录 D 则简要介绍 UML。

对于这本经典之作，译者在翻译过程中力求忠于原文，再现原文风格，但是鉴于译者水平有限，错误和失误在所难免，如有任何意见和建议，请不吝指正。除封面署名的译者外，何益枝、吴秀莲、张继春、段治勋、郑玉花、兰翠平、段国鑫也参与了本书的翻译工作。

最后，希望读者通过阅读本书能早日步入 C++ 编程的殿堂，领略 C++ 语言之美！

作者简介

Marc Gregoire 是一名软件工程师，毕业于比利时鲁文大学，拥有计算机科学与工程硕士学位。之后，他在鲁文大学获得人工智能专业的优等硕士学位。完成学业后，他开始为软件咨询公司 Ordina Belgium 工作。他曾在 Siemens 和 Nokia Siemens Networks 为大型电信运营商提供有关在 Solaris 上运行关键 2G 和 3G 软件的咨询服务。这份工作要求与来自南美、美国、欧洲、中东、非洲和亚洲的国际团队合作。Marc 目前担任 Nikon Metrology (www.nikonmetrology.com) 的软件架构师；Nikon Metrology 是 Nikon 的分公司，是领先的精密光学仪器和 3D 扫描软件供应商。

Marc 的主要技术专长是 C/C++，特别是 Microsoft VC++ 和 MFC 框架。他还擅长在 Windows 和 Linux 平台上开发 24×7 小时运行的 C++ 程序，例如 KNX/EIB 家庭自动化监控软件。除了 C/C++ 之外，Marc 还喜欢 C#，并且会用 PHP 创建网页。

2007 年 4 月，他凭借 Visual C++ 方面的专业技能，获得了微软年度 MVP 称号。

Marc 还是比利时 C++ 用户组 (www.becpp.org) 的创始人，是 *C++ Standard Library Quick Reference* (Apress) 一书的作者，以及多家出版社出版的多本书籍的技术编辑，是 CodeGuru 论坛上的活跃分子 (id 为 Marc G)。Marc 还在 www.nuonsoft.com/blog/ 上维护了一个博客，他热爱旅游和烹饪。

技术编辑简介

Peter Van Weert 是一名软件工程师。他的主要爱好和特长是 C++、编程语言、算法和数据结构。他毕业于比利时勒芬大学，勒芬大学的考试团认定 **Peter** 在计算机科学方面获得了全优成绩，因此授予他硕士学位。2010 年，他在勒芬大学获得博士学位，他的博士论文围绕基于规则的编程语言(主要是 Java)的有效编译展开。在攻读博士学位期间，他还担任面向对象的分析与设计、Java 编程和声明性编程语言等课程的助教。

完成学业后，**Peter** 供职于 **Nikon Metrology**，负责利用 C++ 开发 3D 激光扫描软件和点云检测软件。在 2017 年，**Peter** 加盟 **Nobel Biocare** 的数字化牙科软件研发部门。在他的职业生涯中，**Peter** 精通 C++ 开发以及超大型软件库的管理、重构和调试。**Peter** 也熟悉软件开发过程的所有方面，包括功能和技术需求分析，以及基于敏捷和 Scrum 的项目管理和团队管理。

Peter 是比利时 C++ 用户组的成员，并且经常在该用户组发表技术演讲。他曾参与撰写以下两本由 **Apress** 出版的技术书籍：*C++ Standard Library Quick Reference* 和 *Beginning C++ (5th edition)*。

致 谢

在此感谢 John Wiley & Sons 和 Wrox 出版社的编辑和产品团队，感谢他们的支持。特别感谢 Wiley 的执行编辑 Jim Minatel 给我撰写本书的机会，感谢 Wiley 的高级项目编辑 Adaobi Obi Tulton 对这个项目的支持。感谢文稿编辑 Marylouise Wiack 对文字的梳理，保证术语前后一致，以及确保语法正确无误。

特别感谢技术编辑 Peter Van Weert，感谢他一丝不苟地对本书进行审阅。他提出了很多具有建设性的评论和意见，将本书的质量推进到更高水平。

当然，还要感谢我的父母、兄长和妻子给予的支持和关爱，你们的支持对本书的完成至关重要。也衷心感谢 Nikon Metrology 在我完成本项目期间给予的支持。

最后，我要感谢各位亲爱的读者，希望你们通过本书能学习到高级 C++ 软件开发知识。

多年来，C++都是编写性能卓越、功能强大的企业级面向对象程序的事实标准语言。尽管 C++语言已经风靡全球，但这种语言却非常难完全掌握。专业 C++程序员使用一些简单但高效的技术，这些技术并未出现在传统教材中；即使是经验丰富的 C++程序员，也未必完全了解 C++中某些很有用的特性。

编程书籍往往重点描述语言的语法，而不是语言在真实世界中的应用。典型的 C++教材在每一章中介绍语言中的大部分知识，讲解语法并列举例例。本书不遵循这种模式。本书并不讲解语言的大量细节并给出少量真实世界的场景，而是教你如何在真实世界中使用 C++。本书还会讲解一些鲜为人知的让编程更简单的特性，以及区分编程新手和专业程序员的编程技术。

读者对象

就算使用 C++已经多年，也仍可能不熟悉 C++的一些高级特性，或仍不具有使用这门语言的完整能力。也许你编写过实用的 C++代码，但还想学习更多有关 C++中设计和良好编程风格的内容。也许你是 C++新手，想在入门时就掌握“正确”的编程方式。本书能满足上述需求，将你的 C++技能提升到专业水准。

因为本书专注于对 C++具有基本或中等了解水平蜕变为一名专业 C++程序员的过程，所以本书假设你对该语言具有一定程度的认识。第 1 章涵盖 C++的一些基础知识，可以当成复习材料，但是不能替代实际的语言培训和语言使用手册。如果刚开始接触 C++，但有很丰富的 C、Java 或 C#语言经验，那么应该能从第 1 章获得所需的大部分知识。

不管属于哪种情况，都应该具有很好的编程基础。应该知道循环、函数和变量。应该知道如何组织一个程序，而且应该熟悉基本技术，例如递归。应该了解一些常见的数据结构(例如队列)以及有用的算法(例如排序和搜索)。不需要预先了解有关面向对象编程的知识——这是第 5 章讲解的内容。

还应该熟悉开发代码时使用的编译器。稍后将简要介绍 Microsoft Visual C++和 GCC 这两种编译器。要了解其他编译器，请参阅编译器自带的指南。

本书主要内容

阅读本书是学习 C++语言的一种方法，通过阅读本书既能提升编码质量，又能提升编程效率。本书贯穿对 C++17 新特性的讨论。这些新的 C++17 特性并不是分散在各章中，而是穿插于全书，在有必要的情况下，所有例子都已更新为使用这些新特性。

本书不仅讲解 C++语法和语言特性，还强调编程方法论、可重用的设计模式以及良好的编程风格。本书讲解的方法论覆盖整个软件开发过程——从设计和编码，到调试以及团队协作。这种方法可让你掌握 C++语言及其独有特性，还能在大型软件开发中充分利用 C++语言的强大功能。

想象一下有人学习了 C++的所有语法但没有见过一个 C++例子的情形。他所了解的知识会让他处于非常危险的境地。如果没有示例的引导，他可能会认为所有源代码都要放在程序的 `main()` 函数中，还有可能认为所有变量都应该为全局变量——这些都不是良好的编程实践。

专业的 C++程序员除了理解语法外，还要正确理解语言的使用方式。他们知道良好设计的重要性、面向对

象编程的理论以及使用现有库的最佳方式。他们还开发了大量有用的代码并了解可重用的思想。

通过阅读和理解本书的内容,你也能成为一名专业的 C++程序员。你在 C++方面的知识会得到扩充,将接触到鲜为人知和常被误解的语言特性。你还将领略面向对象设计,掌握卓越的调试技能。最重要的或许是,通过阅读本书,你的头脑中有了大量“可重用”思想,可将这种思想贯彻到日常工作中。

有很多好的理由让你努力成为一名专业的 C++程序员,而非只是泛泛了解 C++。了解语言的真正工作原理有助于提升代码质量。了解不同的编程方法论和过程可让你更好地和团队协作。探索可重用的库和常用的设计模式可提升日常工作效率,并帮助避免白费力气地重复工作。所有这些学习课程都在帮助你成为更优秀的程序员,同时成为更有价值的雇员。

本书结构

本书包括 5 部分。

第 I 部分“专业的 C++简介”是 C++基础速成教程,能确保读者掌握 C++的基础知识。在速成教程后,第 I 部分深入讨论字符串和字符串视图的使用,因为字符串在示例中应用广泛。第 I 部分的最后一章介绍如何编写清晰易读的 C++代码。

第 II 部分“专业的 C++软件设计”介绍 C++设计方法论。你会了解到设计的重要性、面向对象方法论和代码重用的重要性。

第 III 部分“专业的 C++编码方法”从专业角度概述 C++技术。你将学习在 C++中管理内存的最佳方式,如何创建可重用的类,以及如何利用重要的语言特性,例如继承。你还会学习这门语言的一些不同寻常之处、输入输出技术、错误处理、字符串本地化和正则表达式的使用,讨论如何实现运算符重载,如何编写模板。这一部分还讲解 C++标准库,包括容器、迭代器和算法。你还会学习 C++标准中的其他一些库,例如处理时间、随机数和文件系统的库。

第 IV 部分“掌握 C++的高级特性”讲解如何最大限度地使用 C++。本书这一部分揭示 C++中神秘的部分,并描述如何使用这些更高级的特性。你将学习如何定制和扩充标准库以满足自己的需求、高级模板编程的细节(包括模板元编程),以及如何通过多线程编程来充分利用多处理器和多核系统。

第 V 部分“C++软件工程”重点介绍如何编写企业级质量的软件。相关的主题如下:当今编程组织使用的工程实践,如何编写高效的 C++代码,软件测试概念(如单元测试和回归测试),C++程序的调试技术,如何在自己的代码中融入设计技术、框架和概念性的面向对象设计模式,跨语言和跨平台代码的解决方案,等等。

本书最后是 4 个附录。附录 A 列出在 C++技术面试中取得成功的指南,附录 B 是带注解的参考文献列表,附录 C 总结 C++标准中的头文件,附录 D 简要介绍 UML(Unified Modeling Language, 统一建模语言)。

本书没有列出 C++中每个类、方法和函数的参考。Peter Van Weert 和 Marc Gregoire 撰写的 *C++ Standard Library Quick Reference* 是 C++标准库提供的所有重要数据结构、算法和函数的浓缩版。附录 B 列出了更多参考资料。下面是两个很好的在线参考。

www.cppreference.com

可使用这个在线参考,也可下载其离线版本,在没有连接到互联网时使用。

www.cplusplus.com/reference/

本书正文中提到“标准库参考资料”时,就是指上述 C++参考资料。

使用本书的条件

要使用本书,只需要一台带有 C++编译器的计算机。本书只关注 C++中的标准部分,而没有任何编译器厂商相关的扩展。

本书包含 C++17 标准引入的新特性。在撰写本书时,有些编译器还不能完全支持 C++17 的所有新特性。

可使用任意 C++ 编译器。如果还没有 C++ 编译器，可下载一个免费的。这有许多选择。例如，对于 Windows，可下载 Microsoft Visual Studio 2017 Community Edition，这个版本免费且包含 Visual C++；对于 Linux，可使用 GCC 或 Clang，它们也是免费的。

下面将简要介绍如何使用 Visual C++ 和 GCC。可参阅相关的编译器文档来了解更多信息。

Microsoft Visual C++

首先需要创建一个项目。启动 VC++，单击 File | New | Project，在左边的项目模板树中选择 Visual C++ | Win32，再在窗口中间的列表中选择 Win32 Console Application(或 Windows Console Application)模板。在底部指定项目的名称、保存位置，单击 OK。

这会打开一个向导，单击 Next 按钮，选择 Console Application 和 Empty Project，再单击 Finish 按钮。注意，你可能看不到向导，具体取决于使用的 VC++ 2017 版本。相反，将自动创建一个新的项目，其中包含 4 个文件：stdafx.h、stdafx.cpp、targetver.h 和 <projectname>.cpp。如果遇到这种情况，而你想要编译源代码文件(取自从配套网站下载的本书源代码压缩文件)，则必须在 Solution Explorer(选择 View | Solution Explorer)中选择这些文件，然后删除它们。

加载新项目后，就会在 Solution Explorer 中看到项目文件列表。如果这个停靠窗口不可见，可选择 View | Solution Explorer。在 Solution Explorer 中右击项目名，再选择 Add | New Item 或 Add | Existing Item，就可以给项目添加新文件或已有文件。

使用 Build | Build Solution 编译代码。没有编译错误后，就可以使用 Debug | Start Debugging 运行了。

如果程序在查看输出之前就退出了，可使用 Debug | Start without Debugging。这会在程序末尾暂停，以便查看输出。

在撰写本书期间，Visual C++ 2017 尚未自动启用 C++17 功能。要启用 C++17 功能，可在 Solution Explorer 窗口中右击项目，然后单击 Properties。在 Properties 窗口中，选择 Configuration Properties | C/C++ | Language，根据使用的 Visual C++ 版本，将 C++ Language Standard 选项设置为 ISO C++17 Standard 或 ISO C++ Latest Draft Standard。仅当项目至少包含一个 .cpp 文件时，才能访问这些选项。

Visual C++ 支持“预编译的头文件”，这个话题超出了本书的讨论范围。通常而言，如果编译器支持的话，建议使用预编译的头文件。但是，从本书网站下载的源代码文件不使用预编译的头文件，因此，只有禁用这项功能才能使这些代码正确编译。在 Solution Explorer 窗口中右击项目，选择 Properties。在 Properties 窗口中，找到 Configuration Properties | C/C++ | Precompiled Headers，将 Precompiled Header 选项设置为 Not Using Precompiled Headers。

GCC

用自己喜欢的任意文本编辑器创建源代码，保存到一个目录下。要编译代码，可打开一个终端，运行如下命令，指定要编译的所有 .cpp 文件：

```
gcc -lstdc++ -std=c++17 -o <executable_name> <source1.cpp> [source2.cpp ...]
```

-std=c++17 用于告诉 GCC 启用 C++17 支持。

例如，可切换到包含代码的目录，运行如下命令来编译第 1 章的 AirlineTicket 示例：

```
gcc -lstdc++ -std=c++17 -o AirlineTicket AirlineTicket.cpp AirlineTicketTest.cpp
```

没有编译错误后，就可以使用如下命令运行了：

```
./AirlineTicket
```

勘误表

尽管我们已经尽了各种努力来保证文章或代码中不出现错误，但错误总是难免的。如果在本书中找到错误，例如拼写错误或代码错误，请告诉我们，我们将非常感激。通过勘误表，可以让其他读者避免受挫，当然，这还有助于提供更高质量的信息。

请给 wkservice@vip.163.com 发电子邮件，我们会检查你提供的信息，如果是正确的，我们将在本书的后续版本中采用。

要在网站上找到本书的勘误表，可登录 <http://www.wrox.com>，通过 Search 工具或书名列表查找本书，然后在本书的细目页面上，单击 Book Errata 链接。在这个页面上可查看 Wrox 编辑已提交和粘贴的所有勘误项。完整的图书列表还包括每本书的勘误表，网址是 www.wrox.com/misc-pages/booklist.shtml。

源代码

读者在学习本书中的示例时，可以手动输入所有代码，也可使用本书附带的源代码文件。本书使用的所有源代码都可以从本书合作站点 www.wiley.com/go/proc++4e 下载。

另外，也可进入 <http://www.wrox.com/dynamic/books/download.aspx> 上的 Wrox 代码下载主页，查看本书和其他 Wrox 图书的所有代码。

还可通过扫描本书封底的二维码来下载源代码。

提示：

由于许多图书的书名都十分类似，因此按 ISBN 搜索是最简单的，本书英文版的 ISBN 是 978-1-119-42130-6。

下载代码后，只需要用自己喜欢的解压缩软件进行解压缩即可。

目 录

第 I 部分 专业的 C++ 简介

第 1 章 C++ 和标准库速成	2
1.1 C++ 基础知识	2
1.1.1 小程序 “hello world”	3
1.1.2 名称空间	5
1.1.3 字面量	6
1.1.4 变量	7
1.1.5 运算符	8
1.1.6 类型	10
1.1.7 条件语句	12
1.1.8 逻辑比较运算符	14
1.1.9 函数	15
1.1.10 C 风格的数组	16
1.1.11 <code>std::array</code>	17
1.1.12 <code>std::vector</code>	17
1.1.13 结构化绑定	18
1.1.14 循环	18
1.1.15 初始化列表	19
1.1.16 这些都是基础	19
1.2 深入研究 C++	20
1.2.1 C++ 中的字符串	20
1.2.2 指针和动态内存	20
1.2.3 <code>const</code> 的多种用法	24
1.2.4 引用	24
1.2.5 异常	25
1.2.6 类型推断	26
1.3 作为面向对象语言的 C++	27
1.3.1 定义类	27
1.3.2 使用类	29
1.4 统一初始化	29
1.5 标准库	31
1.6 第一个有用的 C++ 程序	31
1.6.1 雇员记录系统	32
1.6.2 <code>Employee</code> 类	32
1.6.3 <code>Database</code> 类	34

1.6.4 用户界面	36
1.6.5 评估程序	38
1.7 本章小结	38
第 2 章 使用 <code>string</code> 和 <code>string_view</code>	39
2.1 动态字符串	39
2.1.1 C 风格的字符串	39
2.1.2 字符串字面量	41
2.1.3 C++ <code>std::string</code> 类	42
2.1.4 <code>std::string_view</code> 类	46
2.1.5 非标准字符串	47
2.2 本章小结	47
第 3 章 编码风格	48
3.1 良好外观的重要性	48
3.1.1 事先考虑	48
3.1.2 良好风格的元素	49
3.2 为代码编写文档	49
3.2.1 使用注释的原因	49
3.2.2 注释的风格	52
3.3 分解	55
3.3.1 通过重构分解	56
3.3.2 通过设计来分解	56
3.3.3 本书中的分解	56
3.4 命名	56
3.4.1 选择恰当的名称	57
3.4.2 命名约定	57
3.5 使用具有风格的语言特性	59
3.5.1 使用常量	59
3.5.2 使用引用代替指针	59
3.5.3 使用自定义异常	59
3.6 格式	60
3.6.1 关于大括号对齐的争论	60
3.6.2 关于空格和圆括号的争论	61
3.6.3 空格和制表符	61
3.7 风格的挑战	61

3.8 本章小结	62
----------------	----

第 II 部分 专业的 C++ 软件设计

第 4 章 设计专业的 C++ 程序	64
4.1 程序设计概述	64
4.2 程序设计的重要性	65
4.3 C++ 设计的特点	66
4.4 C++ 设计的两个原则	67
4.4.1 抽象	67
4.4.2 重用	68
4.5 重用代码	69
4.5.1 关于术语的说明	69
4.5.2 决定是否重用代码	70
4.5.3 重用代码的策略	71
4.5.4 绑定第三方应用程序	74
4.5.5 开放源代码库	75
4.5.6 C++ 标准库	76
4.6 设计一个国际象棋程序	76
4.6.1 需求	76
4.6.2 设计步骤	77
4.7 本章小结	80
第 5 章 面向对象设计	82
5.1 过程化的思考方式	82
5.2 面向对象思想	83
5.2.1 类	83
5.2.2 组件	83
5.2.3 属性	83
5.2.4 行为	84
5.2.5 综合考虑	84
5.3 生活在对象世界里	85
5.3.1 过度使用对象	85
5.3.2 过于通用的对象	85
5.4 对象之间的关系	86
5.4.1 “有一个”关系	86
5.4.2 “是一个”关系(继承)	87
5.4.3 “有一个”与“是一个”的区别	88
5.4.4 not-a 关系	90
5.4.5 层次结构	91
5.4.6 多重继承	91
5.4.7 混入类	92
5.5 抽象	93
5.5.1 接口与实现	93
5.5.2 决定公开的接口	93

5.5.3 设计成功的抽象	94
---------------------	----

5.6 本章小结	95
----------------	----

第 6 章 设计可重用代码	96
---------------------	----

6.1 重用哲学	96
6.2 如何设计可重用代码	97
6.2.1 使用抽象	97
6.2.2 构建理想的重用代码	98
6.2.3 设计有用的接口	102
6.2.4 SOLID 原则	106
6.3 本章小结	106

第 III 部分 专业的 C++ 编码方法

第 7 章 内存管理	108
------------------	-----

7.1 使用动态内存	108
7.1.1 如何描绘内存	109
7.1.2 分配和释放	110
7.1.3 数组	111
7.1.4 使用指针	116
7.2 数组-指针的对偶性	117
7.2.1 数组就是指针	117
7.2.2 并非所有指针都是数组	119
7.3 低级内存操作	119
7.3.1 指针运算	119
7.3.2 自定义内存管理	120
7.3.3 垃圾回收	120
7.3.4 对象池	121
7.4 智能指针	121
7.4.1 unique_ptr	122
7.4.2 shared_ptr	124
7.4.3 weak_ptr	125
7.4.4 移动语义	126
7.4.5 enable_shared_from_this	127
7.4.6 旧的、过时的/取消的 auto_ptr	127
7.5 常见的内存陷阱	127
7.5.1 分配不足的字符串	127
7.5.2 访问内存越界	128
7.5.3 内存泄漏	128
7.5.4 双重删除和无效指针	131
7.6 本章小结	131

第 8 章 熟悉类和对象	132
--------------------	-----

8.1 电子表格示例介绍	132
8.2 编写类	133

8.2.1 类定义	133	10.2 使用继承重用代码	194
8.2.2 定义方法	135	10.2.1 WeatherPrediction类	194
8.2.3 使用对象	137	10.2.2 在派生类中添加功能	195
8.3 对象的生命周期	138	10.2.3 在派生类中替换功能	196
8.3.1 创建对象	138	10.3 利用父类	196
8.3.2 销毁对象	149	10.3.1 父类构造函数	196
8.3.3 对象赋值	149	10.3.2 父类的析构函数	197
8.3.4 编译器生成的复制构造函数和复制赋值 运算符	151	10.3.3 使用父类方法	198
8.3.5 复制和赋值的区别	151	10.3.4 向上转型和向下转型	200
8.4 本章小结	153	10.4 继承与多态性	201
第9章 精通类与对象	154	10.4.1 回到电子表格	201
9.1 友元	154	10.4.2 设计多态性的电子表格单元格	201
9.2 对象的动态内存分配	155	10.4.3 SpreadsheetCell基类	202
9.2.1 Spreadsheet类	155	10.4.4 独立的派生类	203
9.2.2 使用析构函数释放内存	157	10.4.5 利用多态性	204
9.2.3 处理复制和赋值	158	10.4.6 考虑将来	205
9.2.4 使用移动语义处理移动	162	10.5 多重继承	206
9.2.5 零规则	167	10.5.1 从多个类继承	206
9.3 与方法有关的更多内容	167	10.5.2 名称冲突和歧义基类	207
9.3.1 静态方法	167	10.6 有趣而晦涩的继承问题	209
9.3.2 const方法	168	10.6.1 修改重写方法的特征	209
9.3.3 方法重载	169	10.6.2 继承的构造函数	211
9.3.4 内联方法	170	10.6.3 重写方法时的特殊情况	214
9.3.5 默认参数	171	10.6.4 派生类中的复制构造函数和赋值 运算符	219
9.4 不同的数据成员类型	172	10.6.5 运行时类型工具	220
9.4.1 静态数据成员	172	10.6.6 非public继承	221
9.4.2 静态常量数据成员	173	10.6.7 虚基类	221
9.4.3 引用数据成员	174	10.7 本章小结	222
9.4.4 常量引用数据成员	175	第11章 理解灵活而奇特的 C++	223
9.5 嵌套类	175	11.1 引用	223
9.6 类内的枚举类型	176	11.1.1 引用变量	224
9.7 运算符重载	177	11.1.2 引用数据成员	225
9.7.1 示例：为SpreadsheetCell实现加法	177	11.1.3 引用参数	225
9.7.2 重载算术运算符	179	11.1.4 将引用作为返回值	226
9.7.3 重载比较运算符	181	11.1.5 右值引用	226
9.7.4 创建具有运算符重载的类型	181	11.1.6 使用引用还是指针	227
9.8 创建稳定的接口	182	11.2 关键字的疑问	229
9.9 本章小结	184	11.2.1 const关键字	229
第10章 揭秘继承技术	185	11.2.2 static关键字	232
10.1 使用继承构建类	185	11.2.3 非局部变量的初始化顺序	235
10.1.1 扩展类	186	11.2.4 非局部变量的销毁顺序	235
10.1.2 重写方法	188	11.3 类型和类型转换	235

11.3.1	类型别名	235	13.1.2	流的来源和目的地	276
11.3.2	函数指针的类型别名	236	13.1.3	流式输出	277
11.3.3	方法和数据成员的指针的类型别名	238	13.1.4	流式输入	280
11.3.4	typedef	238	13.1.5	对象的输入输出	285
11.3.5	类型转换	239	13.2	字符串流	286
11.4	作用域解析	242	13.3	文件流	287
11.5	特性	243	13.3.1	文本模式与二进制模式	287
11.5.1	[[noreturn]]特性	243	13.3.2	通过seek()和tell()在文件中转移	288
11.5.2	[[deprecated]]特性	244	13.3.3	将流链接在一起	289
11.5.3	[[fallthrough]]特性	244	13.4	双向 I/O	290
11.5.4	[[nodiscard]]特性	244	13.5	本章小结	291
11.5.5	[[maybe_unused]]特性	244	第 14 章	错误处理	292
11.5.6	供应商专用特性	245	14.1	错误与异常	292
11.6	用户定义的字面量	245	14.1.1	异常的含义	292
11.7	头文件	246	14.1.2	C++中异常的优点	293
11.8	C 的实用工具	247	14.1.3	我们的建议	294
11.8.1	变长参数列表	247	14.2	异常机制	294
11.8.2	预处理器宏	249	14.2.1	抛出和捕获异常	295
11.9	本章小结	250	14.2.2	异常类型	296
第 12 章	利用模板编写泛型代码	251	14.2.3	按const和引用捕获异常对象	297
12.1	模板概述	252	14.2.4	抛出并捕获多个异常	297
12.2	类模板	252	14.2.5	未捕获的异常	299
12.2.1	编写类模板	252	14.2.6	noexcept	300
12.2.2	尖括号	258	14.2.7	抛出列表(已不赞成使用/已删除)	300
12.2.3	编译器处理模板的原理	258	14.3	异常与多态性	301
12.2.4	将模板代码分布在多个文件中	258	14.3.1	标准异常体系	301
12.2.5	模板参数	260	14.3.2	在类层次结构中捕获异常	302
12.2.6	方法模板	263	14.3.3	编写自己的异常类	303
12.2.7	类模板的特例化	266	14.3.4	嵌套异常	305
12.2.8	从类模板派生	267	14.4	重新抛出异常	306
12.2.9	继承还是特例化	268	14.5	堆栈的释放与清理	307
12.2.10	模板别名	268	14.5.1	使用智能指针	308
12.3	函数模板	269	14.5.2	捕获、清理并重新抛出	309
12.3.1	函数模板的特例化	270	14.6	常见的错误处理问题	309
12.3.2	函数模板的重载	271	14.6.1	内存分配错误	309
12.3.3	类模板的友元函数模板	271	14.6.2	构造函数中的错误	311
12.3.4	对模板参数推导的更多介绍	272	14.6.3	构造函数的function-try-blocks	312
12.3.5	函数模板的返回类型	272	14.6.4	析构函数中的错误	314
12.4	可变模板	274	14.7	综合应用	315
12.5	本章小结	274	14.8	本章小结	318
第 13 章	C++ I/O 揭秘	275	第 15 章	C++运算符重载	319
13.1	使用流	275	15.1	运算符重载概述	319
13.1.1	流的含义	276	15.1.1	重载运算符的原因	320

15.1.2	运算符重载的限制	320	16.2.6	数学工具	344
15.1.3	运算符重载的选择	320	16.2.7	时间工具	345
15.1.4	不应重载的运算符	322	16.2.8	随机数	345
15.1.5	可重载运算符小结	322	16.2.9	初始化列表	345
15.1.6	右值引用	324	16.2.10	pair和tuple	345
15.1.7	关系运算符	325	16.2.11	optional、variant和any	345
15.2	重载算术运算符	325	16.2.12	函数对象	346
15.2.1	重载一元负号和一元正号运算符	325	16.2.13	文件系统	346
15.2.2	重载递增和递减运算符	326	16.2.14	多线程	346
15.3	重载按位运算符和二元逻辑运算符	327	16.2.15	类型特质	346
15.4	重载插入运算符和提取运算符	327	16.2.16	标准整数类型	346
15.5	重载下标运算符	328	16.2.17	容器	346
15.5.1	通过operator[]提供只读访问	330	16.2.18	算法	351
15.5.2	非整数数组索引	330	16.2.19	标准库中还缺什么	358
15.6	重载函数调用运算符	331	16.3	本章小结	358
15.7	重载解除引用运算符	332	第 17 章	理解容器与迭代器	359
15.7.1	实现operator*	333	17.1	容器概述	359
15.7.2	实现operator>	333	17.1.1	对元素的要求	360
15.7.3	operator.*和operator->的含义	334	17.1.2	异常和错误检查	361
15.8	编写转换运算符	334	17.1.3	迭代器	361
15.8.1	使用显式转换运算符解决 多义性问题	335	17.2	顺序容器	363
15.8.2	用于布尔表达式的转换	335	17.2.1	vector	363
15.9	重载内存分配和内存释放运算符	337	17.2.2	vector<bool>特化	376
15.9.1	new和delete的工作原理	337	17.2.3	deque	377
15.9.2	重载operator new和operator delete	338	17.2.4	list	377
15.9.3	显式地删除/默认化operator new 和operator delete	340	17.2.5	forward_list	380
15.9.4	重载带有额外参数的operator new和 operator delete	340	17.2.6	array	381
15.9.5	重载带有内存大小参数的 operator delete	340	17.3	容器适配器	382
15.10	本章小结	341	17.3.1	queue	382
第 16 章	C++标准库概述	342	17.3.2	priority_queue	384
16.1	编码原则	343	17.3.3	stack	386
16.1.1	使用模板	343	17.4	有序关联容器	387
16.1.2	使用运算符重载	343	17.4.1	pair工具类	387
16.2	C++标准库概述	343	17.4.2	map	388
16.2.1	字符串	343	17.4.3	multimap	393
16.2.2	正则表达式	344	17.4.4	set	396
16.2.3	I/O流	344	17.4.5	multiset	397
16.2.4	智能指针	344	17.5	无序关联容器/哈希表	397
16.2.5	异常	344	17.5.1	哈希函数	397
			17.5.2	unordered_map	399
			17.5.3	unordered_multimap	401
			17.5.4	unordered_set/unordered_multiset	402
			17.6	其他容器	402

17.6.1 标准C风格数组	402	18.6.4 测试auditVoterRolls()函数	443
17.6.2 string	403	18.7 本章小结	444
17.6.3 流	403	第19章 字符串的本地化与正则表达式	445
17.6.4 bitset	403	19.1 本地化	445
17.7 本章小结	407	19.1.1 本地化字符串字面量	445
第18章 掌握标准库算法	408	19.1.2 宽字符	446
18.1 算法概述	408	19.1.3 非西方字符集	446
18.1.1 find()和find_if()算法	409	19.1.4 转换	447
18.1.2 accumulate()算法	410	19.1.5 locale和facet	448
18.1.3 在算法中使用移动语义	411	19.2 正则表达式	450
18.2 std::function	411	19.2.1 ECMAScript语法	451
18.3 lambda 表达式	413	19.2.2 regex库	455
18.3.1 语法	413	19.2.3 regex_match()	456
18.3.2 泛型lambda表达式	415	19.2.4 regex_search()	458
18.3.3 lambda捕捉表达式	415	19.2.5 regex_iterator	458
18.3.4 将lambda表达式用作返回类型	415	19.2.6 regex_token_iterator	459
18.3.5 将lambda表达式用作参数	416	19.2.7 regex_replace()	461
18.3.6 标准库算法示例	416	19.3 本章小结	462
18.4 函数对象	417	第20章 其他库工具	463
18.4.1 算术函数对象	417	20.1 ratio 库	463
18.4.2 比较函数对象	418	20.2 chrono 库	465
18.4.3 逻辑函数对象	419	20.2.1 持续时间	465
18.4.4 按位函数对象	419	20.2.2 时钟	468
18.4.5 函数对象适配器	419	20.2.3 时点	469
18.4.6 std::invoke()	422	20.3 生成随机数	470
18.4.7 编写自己的函数对象	423	20.3.1 随机数引擎	471
18.5 算法详解	423	20.3.2 随机数引擎适配器	472
18.5.1 迭代器	424	20.3.3 预定义的随机数引擎和引擎适配器	472
18.5.2 非修改序列算法	424	20.3.4 生成随机数	473
18.5.3 修改序列算法	427	20.3.5 随机数分布	474
18.5.4 操作算法	433	20.4 optional	476
18.5.5 交换算法	434	20.5 variant	477
18.5.6 分区算法	435	20.6 any	478
18.5.7 排序算法	436	20.7 元组	479
18.5.8 二叉树搜索算法	436	20.7.1 分解元组	481
18.5.9 集合算法	437	20.7.2 串联	481
18.5.10 最大/最小算法	439	20.7.3 比较	482
18.5.11 并行算法	440	20.7.4 make_from_tuple()	482
18.5.12 数值处理算法	440	20.7.5 apply()	483
18.6 算法示例：审核选民登记	442	20.8 文件系统支持库	483
18.6.1 选民登记审核问题描述	442	20.8.1 path	483
18.6.2 auditVoterRolls()函数	442	20.8.2 directory_entry	484
18.6.3 getDuplicates()函数	443	20.8.3 辅助函数	484

20.8.4	目录迭代	484	23.1.2	撕裂	549
20.9	本章小结	485	23.1.3	死锁	549
第 IV 部分 掌握 C++ 的高级特性					
第 21 章	自定义和扩展标准库	488	23.1.4	伪共享	550
21.1	分配器	488	23.2	线程	550
21.2	流适配器	489	23.2.1	通过函数指针创建线程	550
21.2.1	输出流迭代器	489	23.2.2	通过函数对象创建线程	551
21.2.2	输入流迭代器	490	23.2.3	通过 lambda 创建线程	552
21.3	迭代器适配器	490	23.2.4	通过成员函数创建线程	553
21.3.1	反向迭代器	490	23.2.5	线程本地存储	553
21.3.2	插入迭代器	491	23.2.6	取消线程	553
21.3.3	移动迭代器	492	23.2.7	从线程获得结果	553
21.4	扩展标准库	493	23.2.8	复制和重新抛出异常	554
21.4.1	扩展标准库的原因	493	23.3	原子操作库	555
21.4.2	编写标准库算法	493	23.3.1	原子类型示例	556
21.4.3	编写标准库容器	495	23.3.2	原子操作	557
21.5	本章小结	519	23.4	互斥	558
第 22 章	高级模板	520	23.4.1	互斥体类	558
22.1	深入了解模板参数	520	23.4.2	锁	560
22.1.1	深入了解模板类型参数	520	23.4.3	std::call_once	561
22.1.2	template template 参数介绍	522	23.4.4	互斥体对象的用法示例	562
22.1.3	深入了解非类型模板参数	524	23.5	条件变量	565
22.2	模板类部分特例化	525	23.5.1	假唤醒	565
22.3	通过重载模拟函数部分特例化	527	23.5.2	使用条件变量	565
22.4	模板递归	528	23.6	future	566
22.4.1	N 维网格：初次尝试	529	23.6.1	std::promise 和 std::future	567
22.4.2	真正的 N 维网格	529	23.6.2	std::packaged_task	567
22.5	可变参数模板	531	23.6.3	std::async	568
22.5.1	类型安全的变长参数列表	532	23.6.4	异常处理	568
22.5.2	可变数目的混入类	533	23.6.5	std::shared_future	569
22.5.3	折叠表达式	534	23.7	示例：多线程的 Logger 类	570
22.6	模板元编程	535	23.8	线程池	573
22.6.1	编译时阶乘	536	23.9	线程设计和最佳实践	573
22.6.2	循环展开	536	23.10	本章小结	574
22.6.3	打印元组	537	第 V 部分 C++ 软件工程		
22.6.4	类型 trait	539	第 24 章	充分利用软件工程方法	576
22.6.5	模板元编程结论	545	24.1	过程的必要性	576
22.7	本章小结	545	24.2	软件生命周期模型	577
第 23 章	C++ 多线程编程	546	24.2.1	瀑布模型	577
23.1	多线程编程概述	547	24.2.2	生鱼片模型	578
23.1.1	争用条件	548	24.2.3	螺旋类模型	579
			24.2.4	敏捷	581
			24.3	软件工程方法论	581

24.3.1	UP	581	第 27 章	熟练掌握调试技术	625
24.3.2	RUP	582	27.1	调试的基本定律	625
24.3.3	Scrum	583	27.2	bug 分类学	626
24.3.4	极限编程	584	27.3	避免 bug	626
24.3.5	软件分流	587	27.4	为 bug 做好规划	626
24.4	构建自己的过程和方法	588	27.4.1	错误日志	626
24.4.1	对新思想采取开放态度	588	27.4.2	调试跟踪	627
24.4.2	提出新想法	588	27.4.3	断言	633
24.4.3	知道什么行得通、什么行不通	588	27.4.4	崩溃转储	633
24.4.4	不要逃避	588	27.5	静态断言	634
24.5	源代码控制	588	27.6	调试技术	634
24.6	本章小结	590	27.6.1	重现bug	635
第 25 章	编写高效的 C++程序	591	27.6.2	调试可重复的bug	635
25.1	性能和效率概述	591	27.6.3	调试不可重现的bug	635
25.1.1	提升效率的两种方式	592	27.6.4	调试退化	636
25.1.2	两种程序	592	27.6.5	调试内存问题	636
25.1.3	C++是不是低效的语言	592	27.6.6	调试多线程程序	640
25.2	语言层次的效率	592	27.6.7	调试示例：文章引用	640
25.2.1	高效地操纵对象	593	27.6.8	从ArticleCitations示例中总结出的 教训	648
25.2.2	预分配内存	596	27.7	本章小结	649
25.2.3	使用内联方法和函数	596	第 28 章	使用设计技术和框架	650
25.3	设计层次的效率	596	28.1	容易忘记的语法	651
25.3.1	尽可能多地缓存	596	28.1.1	编写类	651
25.3.2	使用对象池	597	28.1.2	派生类	652
25.4	剖析	599	28.1.3	使用“复制和交换”惯用语	652
25.4.1	使用gprof的剖析示例	600	28.1.4	抛出和捕捉异常	653
25.4.2	使用Visual C++ 2017 的剖析示例	605	28.1.5	读取文件	654
25.5	本章小结	608	28.1.6	写入文件	654
第 26 章	熟练掌握测试技术	609	28.1.7	写入模板类	654
26.1	质量控制	609	28.2	始终存在更好的方法	655
26.1.1	谁负责测试	610	28.2.1	RAII	655
26.1.2	bug的生命周期	610	28.2.2	双分派	657
26.1.3	bug跟踪工具	611	28.2.3	混入类	660
26.2	单元测试	612	28.3	面向对象的框架	662
26.2.1	单元测试方法	613	28.3.1	使用框架	662
26.2.2	单元测试过程	613	28.3.2	MVC范型	662
26.2.3	实际中的单元测试	616	28.4	本章小结	663
26.3	高级测试	622	第 29 章	应用设计模式	664
26.3.1	集成测试	622	29.1	迭代器模式	665
26.3.2	系统测试	623	29.2	单例模式	665
26.3.3	回归测试	623	29.2.1	日志记录机制	665
26.4	用于成功测试的建议	624	29.2.2	实现单例	666
26.5	本章小结	624			

29.2.3 使用单例·····	668
29.3 抽象工厂模式·····	668
29.3.1 示例：模拟汽车工厂·····	669
29.3.2 实现工厂·····	670
29.3.3 使用工厂·····	671
29.3.4 工厂的其他用法·····	672
29.4 代理模式·····	672
29.4.1 示例：隐藏网络连接问题·····	673
29.4.2 实现代理·····	673
29.4.3 使用代理·····	674
29.5 适配器模式·····	674
29.5.1 示例：改编Logger类·····	674
29.5.2 实现适配器·····	675
29.5.3 使用适配器·····	675
29.6 装饰器模式·····	675
29.6.1 示例：在网页中定义样式·····	676
29.6.2 装饰器的实现·····	676
29.6.3 使用装饰器·····	677
29.7 责任链模式·····	677
29.7.1 示例：事件处理·····	678
29.7.2 实现责任链·····	678
29.7.3 没有层次结构的责任链·····	679
29.8 观察者模式·····	680
29.8.1 实现观察者·····	680
29.8.2 实现可观察类·····	681
29.8.3 使用观察者·····	681
29.9 本章小结·····	682
第 30 章 开发跨平台和跨语言应用程序·····	683
30.1 跨平台开发·····	683
30.1.1 架构问题·····	683
30.1.2 实现问题·····	686
30.1.3 平台专用功能·····	686
30.2 跨语言开发·····	687
30.2.1 混用C和C++·····	687
30.2.2 改变范型·····	688
30.2.3 链接C代码·····	690
30.2.4 从C#调用C++代码·····	691
30.2.5 在Java中使用JNI调用C++代码·····	692
30.2.6 从C++代码调用脚本·····	694
30.2.7 从脚本调用C++代码·····	694
30.2.8 从C++调用汇编代码·····	696
30.3 本章小结·····	696
附录 A C++面试·····	697
附录 B 带注解的参考文献·····	713
附录 C 标准库头文件·····	721
附录 D UML 简介·····	726

第 I 部分

专业的 C++ 简介

- 第 1 章 C++和标准库速成
- 第 2 章 使用 string 和 string_view
- 第 3 章 编码风格

第 1 章

C++和标准库速成

本章内容

- 简要回顾 C++语言和标准库(Standard Library)最重要的部分以及语法
- 智能指针基础

从 wrox.com 下载本章的示例代码

注意，可访问本书网站 www.wrox.com/go/proc++4e，从 Download Code 选项卡下载本章的所有示例代码。

本章旨在对 C++最重要的部分进行简要描述，使你在阅读本书后面的内容之前掌握一定的基础知识。本章不会全面讲述 C++编程语言或标准库。一些基本知识(如什么是程序和递归)，以及一些深奥的知识(例如，什么是 `union` 和 `volatile` 关键字)也会被忽略。此外还忽略了与 C++关系不大的 C 语言部分，这些内容将在后续章节中详细讨论。

本章主要讲述日常编程中会遇到的那部分 C++知识。例如，如果你有一段时间没有使用 C++，忘了 `for` 循环的语法，你会在本章中找到此语法。此外，如果你刚接触 C++，不了解什么是引用变量，那么可通过阅读本章的内容来了解此类变量的用法。本章还将讲述使用标准库中功能的基础知识，例如 `vector` 容器、`string` 对象和智能指针。

如果你对 C++非常熟悉，请快速浏览本章的内容，看看有没有自己需要复习的某些 C++语言基本内容。如果你刚开始接触 C++，请仔细阅读本章内容并务必理解所有示例。如果需要更多的介绍性信息，请参阅附录 B。

1.1 C++基础知识

C++语言经常被认为是“更好的 C 语言”或“C 语言的超集”，主要被设计为面向对象的 C，常称为“包含类的 C”。后来，在设计 C++时，对 C 语言中许多使用不便或不够精细的内容进行了处理。由于 C++是基于 C 的，如果你有 C 语言编程经验，将发现本节介绍的许多语法非常熟悉。当然这两种语言并不一样，例如，C++语言的创造者 Bjarne Stroustrup 撰写的 *The C++ Programming Language* (Forth Edition; Addison-Wesley Professional, 2013)共有 1368 页，而 Kernighan 和 Ritchie 撰写的 *The C Programming Language* (Second Edition; Prentice Hall, 1988)只有 274 页。因此，如果你是一位 C 程序员，请关注新的或不熟悉的语法。

1.1.1 小程序 “hello world”

下面的代码可能是你所遇到的最简单 C++ 程序：

```
// helloworld.cpp
#include <iostream>

int main()
{
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

正如预期的那样，这段代码会在屏幕上输出 “Hello, World!” 这一信息。这是一个非常简单的程序，好像不会赢得任何赞誉，但是这个程序确实展现出与 C++ 程序格式相关的一些重要概念：

- 注释
- 预处理指令
- main() 函数
- 输入输出流

下面简要描述这些概念。

1. 注释

这个程序的第一行是注释，这只是供编程人员阅读的一条消息，编译器会将其忽略。在 C++ 中，可以通过两种方法添加注释。在前面以及下面的示例中，用两条斜杠表明这一行中在它后面的内容都是注释。

```
// helloworld.cpp
```

使用多行注释也可以实现这个行为(也就是说，二者没什么不同)。多行注释以 /* 开始，以 */ 结束。下面的代码使用了多行注释。

```
/* This is a multiline comment.
   The compiler will ignore it.
*/
```

第 3 章将详细讲述注释。

2. 预处理指令

生成一个 C++ 程序共有三个步骤。首先，代码在预处理器中运行，预处理器会识别代码中的元信息(meta-information)。其次，代码被编译或转换为计算机可识别的目标文件。最后，独立的目标文件被链接在一起变成一个应用程序。

预处理指令以 # 字符开始，前面示例中的 #include <iostream> 行就是如此。在此例中，include 指令告诉预处理器：提取 <iostream> 头文件中的所有内容并提供给当前文件。头文件最常见的用途是声明在其他地方定义的函数。函数声明会通知编译器如何调用这个函数，并声明函数中参数的个数和类型，以及函数的返回类型。而函数定义包含了这个函数的实际代码。在 C++ 中，声明通常放在扩展名为 .h 的文件中，称为头文件，其定义通常包含在扩展名为 .cpp 的文件中，称为源文件。许多其他编程语言(例如 C# 和 Java)不把声明和定义放在不同文件中。

<iostream> 头文件声明了 C++ 提供的输入输出机制。如果程序没有包含这个头文件，甚至无法执行其仅需要完成的输出文本任务。

注意：

在 C 中，标准库头文件的名称通常以 .h 结尾，如 <stdio.h>，不使用名称空间。

在 C++ 中，标准库头文件的名称省略了 .h 后缀，如 <iostream>; 所有文件都在 std 名称空间和 std 的子名称空间中定义。

C 中的标准库头文件在 C++ 中依然存在，但使用以下两个版本。

- 不使用 `.h` 后缀，改用前缀 `c`；这是新版本，也是推荐使用的版本。这些版本将一切放在 `std` 名称空间中(例如 `<cstdio>`)。
- 使用 `.h` 后缀，这是旧版本。这些版本不使用名称空间(例如 `<stdio.h>`)。

表 1-1 给出了最常用的预处理指令。

表 1-1

预处理指令	功能	常见用法
<code>#include [file]</code>	将指定的文件插入代码中指令所在的位置	几乎总是用来包含头文件，使代码可使用在其他位置定义的功能
<code>#define [key] [value]</code>	每个指定的 <code>key</code> 都被替换为指定的值	在 C 中，常用来定义常数值或宏。C++ 提供了常数和大多数宏类型的更好机制。宏的使用具有风险，因此在 C++ 中使用它们要谨慎，更多内容参见第 11 章
<code>#ifdef [key]</code> <code>#endif</code> <code>#ifndef [key]</code> <code>#endif</code>	<code>ifdef</code> ("if defined")块或 <code>ifndef</code> ("if not defined")块中的代码被有条件地包含或舍弃，这取决于是否使用 <code>#define</code> 定义了指定的 <code>key</code>	经常用来防止循环包含。每个头文件都以 <code>#ifndef</code> 开头，以确定缺少一个 <code>key</code> ，然后用一条 <code>#define</code> 指令定义该 <code>key</code> 。头文件以 <code>#endif</code> 结束，这样这个头文件就不会被多次包含。参见该表之后列举的示例
<code>#pragma [xyz]</code>	<code>xyz</code> 因编译器而异。如果在预处理期间执行到这一指令，通常会显示一条警告或错误信息	参见该表之后列举的示例

下面是使用预处理器指令避免重复包含的一个示例：

```
#ifndef MYHEADER_H
#define MYHEADER_H
// ... the contents of this header file
#endif
```

如果编译器支持 `#pragma once` 指令(大多数现代编译器都支持)，可采用以下方法重写上面的代码：

```
#pragma once
// ... the contents of this header file
```

更多内容参见第 11 章。

3. `main()` 函数

`main()` 函数是程序的入口。`main()` 函数返回一个 `int` 值以指示程序的最终执行状态。在 `main()` 函数中，可以忽略显式的 `return` 语句；此时，会自动返回 0。`main()` 函数或者没有参数，或者具有两个参数，如下所示：

```
int main(int argc, char* argv[])
```

`argc` 给出了传递给程序的实参数目，`argv` 包含了这些实参。注意 `argv[0]` 可能是程序的名称，也可能是空字符串，但不应依赖它。相反，应当使用特定于平台的功能来检索程序名。重要的是要记住，实际参数从索引 1 开始。

4. 输入输出流

第 13 章将深入介绍输入输出流，但基本的输入输出非常简单。可将输出流想象为针对数据的洗衣滑槽(chute)，放入其中的任何内容都可以被正确地输出。`std::cout` 就是对应于用户控制台或标准输出的滑槽，此外还有其他滑槽，包括用于输出错误信息的 `std::cerr`。`<<` 运算符将数据放入滑槽，在前面的示例中，引号中的文本字符串被送到标准输出。输出流可在一行代码中连续输出多个不同类型的数据。下面的代码先输出文本，然后是数字，之后是更多文本：

```
std::cout << "There are " << 219 << " ways I love you." << std::endl;
```

`std::endl` 代表序列的结尾。当输出流遇到 `std::endl` 时，就会将已送入滑槽的所有内容输出并转移到下一行。表明一行结尾的另一种方法是使用 `\n` 字符，`\n` 字符是一个转义字符(escape character)，这是一个换行符。转义字符可以在任何被引用的文本字符串中使用。下面列出了最常用的转义字符：

```
\n  换行
\r  回车
\t  制表符
\\  反斜杠字符
\"  引号
```

流还可用于接收用户的输入，最简单的方法是在输入流中使用 `>>` 运算符。`std::cin` 输入流接收用户的键盘输入。下面是一个例子：

```
int value;
std::cin >> value;
```

需要慎重对待用户输入，因为永远都不会知道用户会输入什么类型的数据。第 13 章将全面介绍如何使用输入流。

如果你拥有 C 的背景知识但初次接触 C++，你可能想了解过去使用的、可靠的 `printf()` 和 `scanf()` 现在究竟是什么情况。尽管在 C++ 中仍然使用 `printf()`，但建议你改用流库，主要原因是 `printf()` 和 `scanf()` 未提供类型安全。

1.1.2 名称空间

名称空间用来处理不同代码段之间的名称冲突问题。例如，你可能编写了一段代码，其中有一个名为 `foo()` 的函数。某天，你决定使用第三方库，其中也有一个 `foo()` 函数。编译器无法判断你的代码要使用哪个版本的 `foo()` 函数。库函数的名称无法改变，而改变自己的函数名称又会感到非常痛苦。

在此类情况下可使用名称空间，从而指定定义名称的环境。为将某段代码加入名称空间，可用 `namespace` 块将其包含。假定下面的代码在文件 `namespaces.h` 中：

```
namespace mycode {
    void foo();
}
```

在名称空间中还可实现方法或函数。例如，`foo()` 函数可在 `namespaces.cpp` 中实现，如下所示：

```
#include <iostream>
#include "namespaces.h"
```

```
void mycode::foo()
{
    std::cout << "foo() called in the mycode namespace" << std::endl;
}
```

或者：

```
#include <iostream>
#include "namespaces.h"
```

```
namespace mycode {
    void foo()
    {
        std::cout << "foo() called in the mycode namespace" << std::endl;
    }
}
```

将你编写的 `foo()` 版本放到名称空间 `mycode` 后，这个函数就与第三方库提供的 `foo()` 函数区分开来。为调用启用了名称空间的 `foo()` 版本，需要使用 `::` 在函数名称之前给出名称空间，`::` 称为作用域解析运算符：

```
mycode::foo();    // Calls the "foo" function in the "mycode" namespace
```

`mycode` 名称空间中的任何代码都可调用该名称空间内的其他代码，而不需要显式说明该名称空间。隐式的名称空间可使得代码清晰并易于阅读。可使用 `using` 指令避免预先指明名称空间。这个指令通知编译器，后面的代码将使用指定名称空间中的名称。下面的代码中就隐含了名称空间：

```
#include "namespaces.h"

using namespace mycode;

int main()
{
    foo(); // Implies mycode::foo();
    return 0;
}
```

一个源文件中可包含多条 `using` 指令；这种方法虽然便捷，但注意不要过度使用。极端情况下，如果你使用了已知的所有名称空间，实际上就是完全取消了名称空间。如果使用了两个同名的名称空间，将再次出现名称冲突问题。另外，应该知道代码在哪个名称空间内运行，这样就不会无意中调用错误版本的函数。

前面已经看到了名称空间的语法——在 `hello world` 程序中已经使用过名称空间，`cout` 以及 `endl` 实际上是定义在 `std` 名称空间中的名称。可使用 `using` 指令重新编写 `hello world` 程序，如下所示：

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello, World!" << endl;
    return 0;
}
```

还可以使用 `using` 指令来引用名称空间内的特定项。例如，如果只想使用 `std` 名称空间中的 `cout`，可以这样引用：

```
using std::cout;

后面的代码可使用 cout 而不需要预先指明这个名称空间，但仍然需要显式说明 std 名称空间中的其他项：

using std::cout;
cout << "Hello, World!" << std::endl;
```

警告：

切勿在头文件中使用 `using` 指令或 `using` 声明，否则添加你的头文件的每个人都必须使用它。

C++17

C++17 允许方便地使用嵌套的名称空间，即将一个名称空间放在另一个名称空间中。在 C++17 之前，必须按如下方式使用嵌套的名称空间：

```
namespace MyLibraries {
    namespace Networking {
        namespace FTP {
            /* ... */
        }
    }
}
```

这在 C++17 中得到极大简化：

```
namespace MyLibraries::Networking::FTP {
    /* ... */
}
```

可使用名称空间别名，为另一个名称空间指定一个更简短的新名称。例如：

```
namespace MyFTP = MyLibraries::Networking::FTP;
```

1.1.3 字面量

字面量用于在代码中编写数字或字符串。C++ 支持大量标准字面量。可使用以下字面量指定数字(列出的示例都表示数字 123)：

- 十进制字面量 123
- 八进制字面量 0173

- 十六进制字面量 0x7B
 - 二进制字面量 0b1111011
- C++中的其他字面量示例包括：

- 浮点值(如 3.14f)
- 双精度浮点值(如 3.14)
- 单个字符(如'a')
- 以零结尾的字符数组(如"character array")

还可以定义自己的自变量类型，这是一项高级功能，在第 11 章中介绍。
可以在数值字面量中使用数字分隔符。数字分隔符是一个单引号。例如：

- 23'456'789
- 0.123'456f



C++17 还增加了对十六进制浮点字面量的支持，如 0x3.ABCp-10、0Xb.cp12l。

1.1.4 变量

在 C++中，可在任何位置声明变量，并且可在声明一个变量所在行之后的任意位置使用该变量。声明变量时可不指定值，这些未初始化的变量通常会被赋予一个半随机值，这个值取决于当时内存中的内容(这是许多 bug 的来源)。在 C++中，也可在声明变量时为变量指定初始值。下面的代码给出了两种风格的变量声明方式，使用的都是代表整数值 的 int 类型：

```
int uninitializedInt;
int initializedInt = 7;
cout << uninitializedInt << " is a random value" << endl;
cout << initializedInt << " was assigned an initial value" << endl;
```

注意：
当代码中使用了未初始化的变量时，多数编译器会给出警告或报错信息。当访问未初始化的变量时，某些 C++环境可能会报告运行时错误。

表 1-2 列出了 C++中最常见的变量类型。

表 1-2

类型	说明	用法
(signed) int signed	正整数或负整数，范围取决于编译器(通常是 4 字节)	int i = - 7; signed int i = - 6; signed i = - 5;
(signed) short (int)	短整型整数(通常是 2 字节)	short s = 13; short int s = 14; signed short s = 15; signed short int s = 16;
(signed) long (int)	长整型整数(通常是 4 字节)	long l = - 7L;
(signed) long long (int)	超长整型整数，范围取决于编译器， 但不低于长整型(通常是 8 字节)	long long ll = 14LL;

(续表)

类型	说明	用法
unsigned (int) unsigned short (int) unsigned long (int) unsigned long long (int)	对前面的类型加以限制，使其值>=0	unsigned int i = 2U; unsigned j = 5U; unsigned short s = 23U; unsigned long l = 5400UL; unsigned long long ll = 140ULL;
float	浮点型数字	float f = 7.2f;
double	双精度数字，精度不低于 float	double d = 7.2;
long double	长双精度数字，精度不低于 double	long double d = 16.98L;
char	单个字符	char ch = 'm';
char16_t	单个 16 位字符	char16_t c16 = u'm';
char32_t	单个 32 位字符	char32_t c32 = U'm';
wchar_t	单个宽字符，大小取决于编译器	wchar_t w = L'm';
bool	布尔类型，取值为 true 或 false	bool b = true;
 std::byte ^①	单个字节。在 C++17 之前，字符或无符号字符用于表示一个字节，但那些类型使得像在处理字符。std::byte 却能指明意图，即内存中的单个字节	std::byte b{42}; ^②

注：① 需要<cstdint>头文件的 include 指令。
② std::byte 的初始化需要使用单元列表进行直接列表初始化。

注意：

C++没有提供基本的字符串类型。但是作为标准库的一部分提供了字符串的标准实现，本章后面的内容以及第 2 章将讲述这一问题。

可使用类型转换方式将变量转换为其他类型。例如，可将 float 转换为 int。C++提供了三种方法来显式地转换变量类型。第一种方法来自 C，并且仍然被广泛使用，但实际上，已经不建议使用这种方法；第二种方法初看上去更自然，但很少使用；第三种方法最复杂，却最整洁，是推荐方法。

```
float myFloat = 3.14f;  
int i1 = (int)myFloat;           // method 1  
int i2 = int(myFloat);           // method 2  
int i3 = static_cast<int>(myFloat); // method 3
```

得到的整数是去掉小数部分的浮点数。第 11 章将详细描述这些转换方法之间的区别。在某些环境中，可自动执行类型转换或强制执行类型转换。例如，short 可自动转换为 long，因为 long 代表精度更高的相同数据类型。

```
long someLong = someShort;           // no explicit cast needed
```

当自动转换变量的类型时，应该了解潜在的数据丢失情况。例如，将 float 转换为 int 会丢掉一些信息(数字的小数部分)。如果将一个 float 赋给一个 int 而不显式执行类型转换，多数编译器会给出警告信息。如果确信左边的类型与右边的类型完全兼容，那么隐式转换也完全没有问题。

1.1.5 运算符

如果无法改变变量的值，那么变量还有什么用呢？表 1-3 显示了 C++中最常用的运算符以及使用这些运算符的示例代码。注意，在 C++中，运算符可以是二元的(操作两个表达式)、一元的(仅操作一个表达式)甚至是三

元的(操作三个表达式)。在 C++中只有一个条件运算符，1.1.7 节将介绍这个运算符。

表 1-3

运算符	说明	用法
=	二元运算符，将右边的值赋给左边的变量	int i; i = 3; int j; j = i;
!	一元运算符，改变变量的 true/false(非零或零)状态	bool b = !true; bool b2 = !b;
+	执行加法的二元运算符	int i = 3 + 2; int j = i + 5; int k = i + j;
- * /	执行减法、乘法以及除法的二元运算符	int i = 5 - 1; int j = 5*2; int k = j / i;
%	二元运算符，求除法操作的余数，也称为 mod 运算符	int remainder = 5 % 2;
++	一元运算符，使变量值增 1。如果运算符在变量之后(后增量)，表达式的结果是没有增加的值；如果运算符在变量之前(前增量)，表达式的结果是增 1 后的新值	i++; ++i;
--	一元运算符，使变量值减 1	i--; --i;
+=	i=i+j; 的简写	i+=j;
-=	i=i-j; 的简写	i-=j;
*=	i=i*j; 的简写	i*=j;
/=	i=i/j; 的简写	i/=j;
%=	i=i%j; 的简写	i%=j;
& &=	将一个变量的原始位与另一个变量执行按位“与”运算	i = j & k; j &= k;
 =	将一个变量的原始位与另一个变量执行按位“或”运算	i = j k; j = k;
<< >> <<= >>=	对一个变量的原始位执行移位运算，每一位左移(<<)或右移(>>)指定的位数	i = i << 1; i = i >> 4; i <<= 1; i >>= 4;
^ ^=	执行两个变量之间的按位“异或”运算	i = i ^ j; i ^= j;

下面的程序显示了最常见的变量以及运算符的用法。如果还不确定变量以及运算符的使用方式，请试着判断一下该程序的输出，然后运行该程序来验证自己的答案是否正确。

```
int someInteger = 256;  
short someShort;  
long someLong;  
float someFloat;  
double someDouble;
```

```
someInteger++;
someInteger *= 2;
someShort = static_cast<short>(someInteger);
someLong = someShort * 10000;
someFloat = someLong + 0.785f;
someDouble = static_cast<double>(someFloat) / 100000;
cout << someDouble << endl;
```

关于表达式求值顺序，C++编译器有一套准则。如果某行代码非常复杂，包含多个运算符，其执行顺序可能不会一目了然。因此，最好将复杂表达式分成若干短小的表达式，或使用括号明确地将子表达式分组。例如，除非你记住了 C++运算符的优先级表格，否则下面的代码将比较难懂：

```
int i = 34 + 8 * 2 + 21 / 7 % 2;
```

添加括号可清楚地表明首先执行哪个运算：

```
int i = 34 + (8 * 2) + ( (21 / 7) % 2 );
```

如果测试这些代码，那么这两种方法是等价的，其结果都是 *i* 等于 51。如果你认为 C++按从左到右的顺序对表达式求值，答案将是 1。实际上，C++会首先对/、*以及%求值(从左向右)，然后才执行加减运算，最后是位运算。通过使用圆括号，可明确告诉编译器某个运算应该单独求值。

1.1.6 类型

在 C++中，可使用基本类型(int、bool 等)创建更复杂的自定义数据类型。一旦熟悉 C++程序，就会很少使用以下从 C 沿袭而来的技巧，因为类的功能更强大。虽然如此，你仍有必要学习这两种最常见的创建类型的方法，了解它们的语法。

1. 枚举类型

整数代表某个数字序列中的值。枚举类型允许你定义自己的序列，这样你声明的变量就只能使用这个序列中的值。例如，在一个国际象棋程序中，可以用 int 代表所有棋子，用常量代表棋子的类型，代码如下所示。代表棋子类型的整数用 const 标记，表明这个值永远不会改变。

```
const int PieceTypeKing = 0;
const int PieceTypeQueen = 1;
const int PieceTypeRook = 2;
const int PieceTypePawn = 3;
//etc.
int myPiece = PieceTypeKing;
```

这种表示法虽然也是正确的，但存在一定的风险。因为棋子只是一个 int，如果另一个程序增加棋子的值，会发生什么？加 1 就可让王变成王后，而这实际上没有意义。更糟糕的是，有人可能将某个棋子的值设置为-1，而这个值并没有对应的常量。

枚举类型通过定义变量的取值范围解决了上述问题。下面的代码声明了一个新类型 PieceType，这个类型具有 4 个可能的值，分别代表 4 种国际象棋棋子：

```
enum PieceType { PieceTypeKing, PieceTypeQueen, PieceTypeRook, PieceTypePawn };
```

事实上，枚举类型只是一个整型值。PieceTypeKing 的实际值是 0。然而，由于定义了 PieceType 类型变量可能具有的值，如果试图对 PieceType 变量执行算术运算或将其作为整数对待，编译器会给出警告或错误信息。下面的代码声明了一个 PieceType 类型的变量，然后试图将其作为整数使用，大多数编译器会给出警告或错误信息：

```
PieceType myPiece;
myPiece = 0;
```

还可为枚举成员指定整型值，其语法如下：

```
enum PieceType { PieceTypeKing = 1, PieceTypeQueen, PieceTypeRook = 10, PieceTypePawn };
```

在此例中，PieceTypeKing 具有整型值 1，编译器为 PieceTypeQueen 赋予整型值 2，PieceTypeRook 的值为 10，编译器自动为 PieceTypePawn 赋予值 11。

编译器会将上一个枚举成员的值递增 1，再赋予当前的枚举成员。如果没有给第一个枚举成员赋值，编译器就给它赋予 0。

2. 强类型枚举

上面给出的枚举并不是强类型的，这意味着其并非类型安全的。它们总被解释为整型数据，因此可以比较完全不同的枚举类型中的枚举值。

强类型的 `enum class` 枚举解决了这些问题，例如，下面定义前述 `PieceType` 枚举的类型安全版本：

```
enum class PieceType
{
    King = 1,
    Queen,
    Rook = 10,
    Pawn
};
```

对于 `enum class`，枚举值名不会自动超出封闭的作用域，这表示总要用作用域解析操作符：

```
PieceType piece = PieceType::King;
```

这也意味着给枚举值指定了更简短的名称，例如，用 `King` 替代 `PieceTypeKing`。

另外，枚举值不会自动转换为整数。因此，下面的代码是不合法的：

```
if (PieceType::Queen == 2) {...}
```

默认情况下，枚举值的基本类型是整型，但可采用以下方式加以改变：

```
enum class PieceType : unsigned long
{
    King = 1,
    Queen,
    Rook = 10,
    Pawn
};
```

注意：

建议用类型安全的 `enum class` 枚举替代类型不安全的 `enum` 枚举。

3. 结构

结构(struct)允许将一个或多个已有类型封装到一个新类型中。数据库记录是结构的经典示例，如果想要建立一个人事系统来跟踪雇员的信息，那么需要存储名首字母、姓首字母、雇员编号以及每个雇员的薪水。下面的代码给出了 `employeestruct.h` 头文件中的一个结构，这个结构包含所有这些信息：

```
struct Employee {
    char firstInitial;
    char lastInitial;
    int employeeNumber;
    int salary;
};
```

声明为 `Employee` 类型的变量将拥有全部内建的字段。可使用运算符访问结构的各个字段。下面的示例创建了一条员工记录，然后将其输出：

```
#include <iostream>
#include "employeestruct.h"

using namespace std;

int main()
{
    // Create and populate an employee.
    Employee anEmployee;
    anEmployee.firstInitial = 'M';
    anEmployee.lastInitial = 'G';
    anEmployee.employeeNumber = 42;
    anEmployee.salary = 80000;
    // Output the values of an employee.
```



```

    cout << "Employee: " << anEmployee.firstInitial <<
        anEmployee.lastInitial << endl;
    cout << "Number: " << anEmployee.employeeNumber << endl;
    cout << "Salary: $" << anEmployee.salary << endl;
    return 0;
}

```

1.1.7 条件语句

条件语句允许根据某件事情的真假来执行代码。在下面介绍的内容中，你可以了解到 C++ 中有三种主要的条件语句：if/else 语句、switch 语句和条件运算符。

1. if/else 语句

最常见的条件语句是 if 语句，其中可能伴随着 else 语句。如果 if 语句中给定的条件为 true，就执行对应的代码行或代码块，否则执行 else 语句(如果存在 else 语句)或者执行条件语句之后的代码。下面的伪代码显示了一种级联的 if 语句，这是一种奇特方式：if 语句伴随着 else 语句，而 else 语句又伴随着另一个 if 语句。

```

if (i > 4) {
    // Do something.
} else if (i > 2) {
    // Do something else.
} else {
    // Do something else.
}

```

if 语句的圆括号中的表达式必须是一个布尔值，或者求值的结果必须是布尔值。零值是 false，非零值则算作 true。例如，if(0) 等同于 if(false)。后面将讲到的逻辑条件运算符提供了一种求表达式值的方式，其结果为布尔值 true 或 false。

C++17

2. if 语句的初始化器

C++17 允许在 if 语句中包括一个初始化器，语法如下：

```
if (<initializer> ; <conditional_expression>) { <body> }
```

<initializer> 中引入的任何变量只在 <conditional_expression> 和 <body> 中可用。此类变量在 if 语句以外不可用。

此时还不到列举此功能有用示例的时候，下面只列出它的形式：

```
if (Employee employee = GetEmployee() ; employee.salary > 1000) { ... }
```

在这个示例中，初始化器获得一名雇员，以及检查所检索雇员的薪水是否超出 1000 的条件。只有满足条件才执行 if 语句体。

本书将穿插列举具体示例。

3. switch 语句

switch 是另一种根据表达式值执行操作的语法。在 C++ 中，switch 语句的表达式必须是整型、能转换为整型的类型、枚举类型或强类型枚举，必须与一个常量进行比较，每个常量值代表一种“情况(case)”，如果表达式与这种情况匹配，随后的代码行将会被执行，直至遇到 break 语句为止。此外还可提供 default 情况，如果没有其他情况与表达式值匹配，表达式值将与 default 情况匹配。下面的伪代码显示了 switch 语句的常见用法：

```

switch (menuItem) {
    case OpenMenuItem:
        // Code to open a file
        break;
    case SaveMenuItem:
        // Code to save a file
        break;
    default:
        // Code to give an error message
        break;
}

```

switch 语句总可转换为 if/else 语句。前面的 switch 语句可以转换为：

```
if (menuItem == OpenMenuItem) {
    // Code to open a file
} else if (menuItem == SaveMenuItem) {
    // Code to save a file
} else {
    // Code to give an error message
}
```

如果你想基于表达式的多个值(而非对表达式进行一些测试)执行操作，通常使用 switch 语句。此时，switch 语句可避免级联使用 if-else 语句。如果只需要检查一个值，则应当使用 if 或 if-else 语句。

一旦找到与 switch 条件匹配的 case 表达式，就执行其后的所有语句，直至遇到 break 语句为止。即使遇到另一个 case 表达式，执行也会继续，这称为 fallthrough。下例有一组语句，会为不同的 case 执行：

```
switch (backgroundColor) {
    case Color::DarkBlue:
    case Color::Black:
        // Code to execute for both a dark blue or black background color
        break;
    case Color::Red:
        // Code to execute for a red background color
        break;
}
```

 如果你无意间忘掉了 break 语句，fallthrough 将成为 bug 的来源。因此，如果在 switch 语句中检测到 fallthrough，编译器将生成警告信息，除非像上例那样 case 为空。从 C++17 开始，你可以使用[[fallthrough]]特性，告诉编译器某个 fallthrough 是有意为之，如下所示：

```
switch (backgroundColor) {
    case Color::DarkBlue:
        doSomethingForDarkBlue();
        [[fallthrough]];
    case Color::Black:
        // Code is executed for both a dark blue or black background color
        doSomethingForBlackOrDarkBlue();
        break;
    case Color::Red:
    case Color::Green:
        // Code to execute for a red or green background color
        break;
}
```

4. switch 语句的初始化器

与 if 语句一样，C++17 支持在 switch 语句中使用初始化器。语法如下：

```
switch (<initializer> ; <expression>) { <body> }
```

<initializer>中引入的任何变量将只在<expression>和<body>中可用。它们在 switch 语句之外不可用。

5. 条件运算符

C++有一个接收三个参数的运算符，称为三元运算符。可将其作为“如果[某事发生了]，那么[执行某个操作]，否则[执行其他操作]”的条件表达式的简写。这个条件运算符由一个?以及一个:组成。下面的代码中，如果变量 i 的值大于 2，将输出 yes，否则输出 no：

```
std::cout << ((i > 2) ? "yes" : "no");
```

i > 2 两边的小括号是可选的，因此与下面的代码行是等效的：

```
std::cout << (i > 2 ? "yes" : "no");
```

条件运算符的优点是几乎可在任何环境中使用。在上例中，这个条件运算符在输出语句中执行。记住这个语法的简便方法是将问号前的语句真的当作一个问题。例如，“i 大于 2 吗？如果是真的，结果就是 yes，否则结果就是 no。”

与 if 语句以及 switch 语句不同，条件运算符不会根据结果执行代码块，而是用在代码中，如上例所示。在

这种用法中，它其实是一个运算符(类似于+和-)，而不是条件语句(例如 if 以及 switch)。

1.1.8 逻辑比较运算符

前面介绍了非正式定义的逻辑比较运算符(conditional operator)。>运算符比较两个值的大小，如果左边的值大于右边的值，那么结果为 true。所有逻辑比较运算符都遵循这一模式——其结果都是 true 或 false。

表 1-4 列出了常见的逻辑比较运算符。

表 1-4

运算符	说明	用法
< <= > >=	判断左边的值是否小于、小于或等于、大于、大于或等于右边的值	<pre>if (i < 0) { std::cout << "i is negative"; }</pre>
==	判断左边的值是否等于右边的值,不要将其与赋值运算符=混淆	<pre>if (i == 3) { std::cout << "i is 3"; }</pre>
!=	不等于。如果左边的值与右边的值不相等，则语句的结果为 true	<pre>if (i != 3) { std::cout << "i is not 3"; }</pre>
!	逻辑非。改变布尔表达式的 true/false 状态，这是一个一元运算符	<pre>if (!someBoolean) { std::cout << "someBoolean is false"; }</pre>
&&	逻辑与。如果表达式的两边都为 true，其结果为 true	<pre>if (someBoolean && someOtherBoolean) { std::cout << "both are true"; }</pre>
	逻辑或。如果表达式两边的任意一边值为 true，其结果就为 true	<pre>if (someBoolean someOtherBoolean) { std::cout << "at least one is true"; }</pre>

C++对表达式求值时会采用短路逻辑。这意味着一旦最终结果可确定，就不对表达式的剩余部分求值。例如，当执行如下所示的多个布尔表达式的逻辑或操作时，如果发现其中一个表达式的值为 true，立刻可判定其结果为 true，就不再检测剩余部分。

```
bool result = bool1 || bool2 || (i > 7) || (27 / 13 % i + 1) < 2;
```

在此例中，如果 bool1 的值是 true，整个表达式的值必然为 true，因此不会对其他部分求值。这种方法可阻止代码执行多余操作。然而，如果后面的表达式以某种方式影响程序的状态(例如，调用了一个独立函数)，就会带来难以发现的 bug。下面的代码显示了一条使用了&&的语句，这条语句在第二项之后就会被短路，因为 0 总被当作 false:

```
bool result = bool1 && 0 && (i > 7) && !done;
```

短路做法对性能有好处。在使用逻辑短路时，可将代价更低的测试放在前面，以避免执行代价更高的测试。在指针上下文中，它也可避免指针无效时执行表达式的一部分的情况。本章后面将讨论指针以及包含短路的指针。

1.1.9 函数

对于任何大型程序而言，将所有代码都放到 `main()` 中都是无法管理的。为使程序便于理解，需要将代码分解为简单明了的函数。

在 C++ 中，为让其他代码使用某个函数，首先应该声明该函数。如果函数在某个特定的文件内部使用，通常会在源文件中声明并定义这个函数。如果函数是供其他模块或文件使用的，通常会在头文件中声明函数，并在源文件中定义函数。

注意：

函数声明通常称为“函数原型”或“函数头”，以强调这代表函数的访问方式，而不是具体代码。术语“函数签名”指将函数名与形参列表组合在一起，但没有返回类型。

函数的声明如下所示。这个示例的返回值是 `void` 类型，说明这个函数不会向调用者提供结果。调用者在调用函数时必须提供两个参数——一个整数和一个字符。

```
void myFunction(int i, char c);
```

如果没有与函数声明匹配的函数定义，在编译过程的链接阶段会出错，因为使用函数 `myFunction()` 时会调用不存在的代码。下面的函数定义输出了两个参数的值：

```
void myFunction(int i, char c)
{
    std::cout << "the value of i is " << i << std::endl;
    std::cout << "the value of c is " << c << std::endl;
}
```

在程序的其他位置，可调用 `myFunction()`，并将实参传递给两个形参。函数调用的几个示例如下：

```
myFunction(8, 'a');
myFunction(someInt, 'b');
myFunction(5, someChar);
```

注意：

与 C 不同，在 C++ 中没有形参的函数仅需要一个空的参数列表，不需要使用 `void` 指出此处没有形参。然而，如果没有返回值，仍需要使用 `void` 来指明这一点。

C++ 函数还向调用者返回一个值。下面的函数将两个数字相加并返回结果：

```
int addNumbers(int number1, int number2)
{
    return number1 + number2;
}
```

这个函数的调用如下：

```
int sum = addNumbers(5, 3);
```

1. 函数返回类型的推断

C++14 允许要求编译器自动推断出函数的返回类型。要使用这个功能，需要把 `auto` 指定为返回类型：

```
auto addNumbers(int number1, int number2)
{
    return number1 + number2;
}
```

编译器根据 `return` 语句使用的表达式推断返回类型。函数中可有多个 `return` 语句，但它们应解析为相同的类型。这种函数甚至可包含递归调用(调用自身)，但函数中的第一个 `return` 语句必须是非递归调用。

2. 当前函数的名称

每个函数都有一个预定义的局部变量 `__func__`，其中包含当前函数的名称。这个变量的一个用途是用于日

志记录:

```
int addNumbers(int number1, int number2)
{
    std::cout << "Entering function " << __func__ << std::endl;
    return number1 + number2;
}
```

1.1.10 C 风格的数组

数组具有一系列值，所有值的类型相同，每个值都可根据它在数组中的位置进行访问。在 C++ 中声明数组时，必须声明数组的大小。数组的大小不能用变量表示——必须用常量或常量表达式(constexpr)表示数组大小，这一内容将在第 11 章中讨论。在下面的代码中，首先声明了具有 3 个整数的数组，之后的三行语句将每个元素初始化为 0:

```
int myArray[3];
myArray[0] = 0;
myArray[1] = 0;
myArray[2] = 0;
```

警告:

在 C++ 中，数组的第一个元素始终在位置 0，而非位置 1！数组的最后一个元素的位置始终是数组大小减 1！

下一节将讨论循环，使用循环可初始化每个元素。但不使用循环或前面的初始化机制，也可以使用如下单行代码完成将所有元素初始化为 0 的操作:

```
int myArray[3] = {0};
```

甚至可以丢弃 0，如下所示:

```
int myArray[3] = {};
```

数组也可以用初始化列表初始化，此时编译器可自动推断出数组的大小。例如:

```
int myArray[] = {1, 2, 3, 4}; // The compiler creates an array of 4 elements.
```

如果指定了数组的大小，而初始化列表包含的元素数量少于给定大小，则将其余元素设置为 0。例如，以下代码仅将数组中的第一个元素设置为 2，将其他元素设置为 0:

```
int myArray[3] = {2};
```

要设置基于栈的 C 风格数组的大小，可使用 C++17 std::size() 函数(需要 <array>)。例如:

```
unsigned int arraySize = std::size(myArray);
```

如果你的编译器与 C++17 不兼容，要获得基于栈的 C 风格数组的大小，旧的做法是使用 sizeof 操作符。sizeof 操作符返回实参的大小(单位为字节)。要获取基于栈的数组中的元素数量，则用数组的字节大小除以第一个元素的字节大小。例如:

```
unsigned int arraySize = sizeof(myArray) / sizeof(myArray[0]);
```

前面的代码显示了一个一维数组，可将其当作一行整数，每个数字都具有自己的编号。C++ 允许使用多维数组，可将二维数组看成西洋跳棋棋盘，每个位置都具有 x 坐标值和 y 坐标值。三维数组和更高维的数组更难描绘，因此极少使用。下面的代码显示了用于井字游戏棋盘的二维字符数组，然后在位于中央的方块中填充一个 o。

```
char ticTacToeBoard[3][3];
ticTacToeBoard[1][1] = 'o';
```

图 1-1 显示了这个棋盘的图形表示，并给出了每个方块的位置。

TicTacToeBoard[0][0]	TicTacToeBoard[0][1]	TicTacToeBoard[0][2]
TicTacToeBoard[1][0]	TicTacToeBoard[1][1]	TicTacToeBoard[1][2]
TicTacToeBoard[2][0]	TicTacToeBoard[2][1]	TicTacToeBoard[2][2]

图 1-1

注意：

在 C++中，最好避免使用本节讨论的 C 风格的数组，而改用标准库功能，如 `std::array` 和 `std::vector`，如以下两节所述。

1.1.11 `std::array`

上一节讨论的数组来自 C，仍能在 C++中使用。但 C++有一种大小固定的特殊容器 `std::array`，这种容器在 `<array>` 头文件中定义。它基本上是对 C 风格的数组进行了简单包装。

用 `std::array` 替代 C 风格的数组会带来很多好处。它总是知道自身大小；不会自动转换为指针，从而避免了某些类型的 bug；具有迭代器，可方便地遍历元素。第 17 章将详细讲述迭代器。

下例演示了 `array` 容器的用法。在 `array<int, 3>` 中，`array` 后面的尖括号在第 12 章中讨论。将尖括号用于 `array`，足以令人记住，必须在尖括号中指定两个参数。第一个参数表示数组中元素的类型，第二个参数表示数组的大小。

```
array<int, 3> arr = {9, 8, 7};
cout << "Array size = " << arr.size() << endl;
cout << "2nd element = " << arr[1] << endl;
```

注意：

C 风格的数组和 `std::array` 都具有固定的大小，在编译时必须知道这一点。在运行时数组不会增大或缩小。

如果希望数组的大小是动态的，推荐使用下一节介绍的 `std::vector`。在 `vector` 中添加新元素时，`vector` 会自动增加其大小。

1.1.12 `std::vector`

标准库提供了多个不同的非固定大小容器，可用于存储信息。`std::vector` 就是此类容器的一个示例，它在 `<vector>` 中声明，用一种更灵活和安全的机制取代 C 中数组的概念。用户不需要担心内存的管理，因为 `vector` 将自动分配足够的内存来存放其元素。`vector` 是动态的，意味着可在运行时添加和删除元素。第 17 章将详细讨论容器，但 `vector` 的基本用法很简单。所以本书一开头就介绍它，以便在示例中使用。下面的示例演示了 `vector` 的基本功能。

```
// Create a vector of integers
vector<int> myVector = { 11, 22 };

// Add some more integers to the vector using push_back()
myVector.push_back(33);
myVector.push_back(44);
```

```
// Access elements
cout << "1st element: " << myVector[0] << endl;
```

将 `myVector` 声明为 `vector<string>`，尖括号用来指定模板参数，与本章前面的 `std::array` 一样。`vector` 是一个泛型容器，几乎可容纳任何类型的对象；但是必须使用尖括号指定要在 `vector` 中存放的对象类型。模板将在第 12 章和第 22 章中详细讨论。

为向 `vector` 中添加元素，可使用 `push_back()` 方法。可使用类似于数组的语法(即 `operator[]`)来访问各个元素。

1.1.13 结构化绑定

C++17 引入了结构化绑定(structured bindings)的概念，允许声明多个变量，这些变量使用数组、结构、`pair` 或元组中的元素来初始化。

例如，假设有下面的数组：

```
std::array<int, 3> values = { 11, 22, 33 };
```

可声明三个变量 `x`、`y` 和 `z`，使用其后数组中的三个值进行初始化。注意，必须为结构化绑定使用 `auto` 关键字。例如，不能用 `int` 替代 `auto`。

```
auto [x, y, z] = values;
```

使用结构化绑定声明的变量数量必须与右侧表达式中的值数量匹配。

如果所有非静态成员都是公有的，也可将结构化绑定用于结构。例如：

```
struct Point { double mX, mY, mZ; };
Point point;
point.mX = 1.0; point.mY = 2.0; point.mZ = 3.0;
auto [x, y, z] = point;
```

第 17 章和第 20 章分别列举了 `std::pair` 和 `std::tuple` 的示例。

1.1.14 循环

计算机擅长重复执行类似的任务。C++ 提供了 4 种循环结构：`while` 循环、`do/while` 循环、`for` 循环和基于区间的 `for` 循环。

1. while 循环

只要条件表达式的求值结果为 `true`，`while` 循环就会重复执行一个代码块。例如，下面的代码会输出 “This is silly.” 5 次：

```
int i = 0;
while (i < 5) {
    std::cout << "This is silly." << std::endl;
    ++i;
}
```

在循环中可使用 `break` 关键字立刻跳出循环并继续执行程序。关键字 `continue` 可用来返回到循环顶部并对 `while` 表达式重新求值。这两种风格都不提倡使用，因为它们会使程序的执行产生无规则的跳转，应该慎用。

2. do/while 循环

C++ 还有一个版本的 `while` 循环，叫作 `do/while` 循环。其运行方式类似于 `while` 循环，但会首先执行代码，而判断是否继续执行的条件检测被放在结尾处。如果想让代码块至少执行一次，并且根据某一条件确定是否多次执行，就可以使用这个循环版本。下面的代码尽管条件为 `false`，但仍会输出 “This is silly.” 一次。

```
int i = 100;
do {
    std::cout << "This is silly." << std::endl;
    ++i;
} while (i < 5);
```

3. for 循环

for 循环提供了另一种循环语法。任何 for 循环都可转换为 while 循环，反之亦然。然而，for 循环的语法一般更简便，因为可看到循环的初始表达式、结束条件以及每次迭代结束后执行的语句。在下面的代码中，i 被初始化为 0；只要 i 小于 5，循环就会继续执行；每次迭代结束时，i 的值会增 1。这段代码的功能与 while 循环示例相同，但这段代码看起来更清晰，因为在一行中显示了初始值、结束条件以及每次迭代结束后执行的语句。

```
for (int i = 0; i < 5; ++i) {
    std::cout << "This is silly." << std::endl;
}
```

4. 基于区间的 for 循环

基于区间的 for 循环(Range-Based for Loop)是第 4 种循环，这种循环允许方便地迭代容器中的元素。这种循环类型可用于 C 风格的数组、初始化列表(稍后讨论)，也可用于具有返回迭代器的 begin()和 end()函数的类型，例如 std::array、std::vector 以及第 17 章讨论的其他所有标准库容器。

下例首先定义一个包含 4 个整数的数组，此后“基于区间的 for 循环”遍历数组中的每个元素，输出每个值。为在迭代元素时不制作副本，应使用本章后面讨论的引用变量：

```
std::array<int, 4> arr = {1, 2, 3, 4};
for (int i : arr) {
    std::cout << i << std::endl;
}
```

1.1.15 初始化列表

初始化列表在<initializer_list>头文件中定义；利用初始化列表，可轻松地编写能接收可变数量参数的函数。initializer_list 类是一个模板，要求在尖括号之间指定列表中的元素类型，这类似于指定 vector 中存储的对象类型。下例演示如何使用初始化列表：

```
#include <initializer_list>

using namespace std;

int makeSum(initializer_list<int> lst)
{
    int total = 0;
    for (int value : lst) {
        total += value;
    }
    return total;
}
```

makeSum()函数接收一个整型类型的初始化列表作为参数。函数体使用“基于区间的 for 循环”来累加总数。可按如下方式使用该函数：

```
int a = makeSum({1,2,3});
int b = makeSum({10,20,30,40,50,60});
```

初始化列表是类型安全的，会定义列表中允许的类型。对于此处的 makeSum()函数，初始化列表中的所有元素都必须是整数。尝试使用 double 数值进行调用，将导致编译器生成错误或警告，如下所示：

```
int c = makeSum({1,2,3.0});
```

1.1.16 这些都是基础

到此为止，你已经回顾了 C++ 程序设计的基本要点。如果这部分内容对你来说非常简单，可以快速浏览下一节以尽快接触更高级的内容。如果觉得这部分内容难以理解，请在学习后面的内容之前阅读附录 B 中建议的入门级 C++ 书籍。

1.2 深入研究 C++

循环、变量以及条件语句都是极好的构件，但需要学习的内容还有很多。下面讲述的内容包含了许多特性，这些特性用于帮助 C++ 程序员编写代码；但有些特性是弊大于利的。如果你是一位几乎没有 C++ 经验的 C 程序员，那么应该仔细阅读本节的内容。

1.2.1 C++ 中的字符串

在 C++ 中使用字符串有三种方法。一种是 C 风格，将字符串看成字符数组；一种是 C++ 风格，将字符串封装到一种易于使用的 `string` 类型中；还有一种是非标准的普通类。第 2 章将详细论述。

现在，只需要知道，C++ `string` 类型在 `<string>` 头文件中定义，C++ `string` 的用法与基本类型几乎相同。与 I/O 流一样，`string` 类型位于 `std` 名称空间中。下面的示例说明了 `string` 如何像字符数组那样使用：

```
string myString = "Hello, World";
cout << "The value of myString is " << myString << endl;
cout << "The second letter is " << myString[1] << endl;
```

1.2.2 指针和动态内存

动态内存允许所创建的程序具有在编译时大小可变的数据，大多数复杂程序都会以某种方式使用动态内存。

1. 堆栈和堆

C++ 程序中的内存分为两个部分——堆栈和堆。堆栈就像一副扑克牌，当前顶部的牌代表程序当前的作用域，通常是当前正在执行的函数。当前函数中声明的所有变量将占用顶部堆栈帧(也就是最上面的那张牌)的内存。如果当前函数(将其称为 `foo()`)调用了另一个函数 `bar()`，就会翻开一张新牌，这样 `bar()` 就会拥有自己的堆栈帧供其运行。任何从 `foo()` 传递给 `bar()` 的参数都会从 `foo()` 堆栈帧复制到 `bar()` 堆栈帧。图 1-2 显示了执行 `foo()` 函数时堆栈的情形，`foo()` 函数中声明了两个整型值。

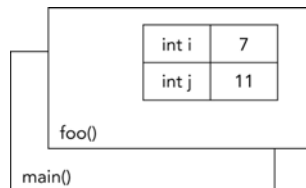


图 1-2

堆栈帧很好，因为它为每个函数提供了独立的内存空间。如果在 `foo()` 堆栈帧中声明了一个变量，那么除非专门要求，否则调用 `bar()` 函数不会更改该变量。此外，`foo()` 函数执行完毕时，堆栈帧就会消失，该函数中声明的所有变量都不会再占用内存。在堆栈上分配内存的变量不需要由程序员释放内存(删除)，这个过程是自动完成的。

堆是与当前函数或堆栈帧完全没有关系的内存区域。如果想在函数调用结束之后仍然保存其中声明的变量，可以将变量放到堆中。堆的结构不如堆栈复杂，可以将堆当作一堆位。程序可在任何时候向堆中添加新位或修改堆中已有的位。必须确保释放(删除)在堆上分配的任何内存，这个过程不会自动完成，除非使用了智能指针(见稍后的“智能指针”小节)。

2. 使用指针

明确地分配内存，就可在堆上放置任何内容。例如，要在堆上放置一个整数，需要给它分配内存，但首先需要声明一个指针：

```
int* myIntegerPointer;
```

`int` 类型后面的*表示，所声明的变量引用/指向某个整数内存。可将指针看成指向动态分配堆上内存的一个箭头，它还没有指向任何内容，因为还可以把它赋予任何内容，它是一个未初始化的变量。在任何时候都应避免使用未初始化的变量，尤其是未初始化的指针，因为它们会指向内存中的某个随意位置。使用这种指针很可能使程序崩溃。这就是应同时声明和初始化指针的原因。如果不希望立即分配内存，可以把它们初始化为空指针 `nullptr` (详见后面的“空指针常量”小节)：

```
int* myIntegerPointer = nullptr;
```

空指针是一个特殊的默认值，有效的指针都不含该值，在布尔表达式中使用时会转换为 `false`。例如：

```
if (!myIntegerPointer) { /* myIntegerPointer is a null pointer */ }
```

使用 `new` 操作符分配内存：

```
myIntegerPointer = new int;
```

在此情况下，指针只是指向一个整数值地址。为访问这个值，需要对指针解除引用。可将解除引用看成沿着指针箭头的方向寻找堆中实际的值。为给堆中新分配的整数赋值，可采用如下代码：

```
*myIntegerPointer = 8;
```

注意，这并非将 `myIntegerPointer` 的值设置为 8，在此并没有改变指针，而是改变了指针所指的内存。如果真要重新设置指针的值，它将指向内存地址 8，这可能是一个随机的无用内存单元，最终会导致程序崩溃。

使用完动态分配的内存后，需要使用 `delete` 操作符释放内存。为防止在释放指针指向的内存后再使用指针，建议把指针设置为 `nullptr`：

```
delete myIntegerPointer;
myIntegerPointer = nullptr;
```

警告：

在解除引用之前指针必须有效。对 `null` 或未初始化的指针解除引用会导致不可确定的行为。程序可能崩溃，也可能继续运行，但开始给出奇怪的结果。

指针并非总是指向堆内存，可声明一个指向堆栈中变量甚至指向其他指针的指针。为让指针指向某个变量，需要使用“取址”运算符 `&`：

```
int i = 8;
int* myIntegerPointer = &i; // Points to the variable with the value 8
```

C++ 使用特殊语法来处理指向结构的指针。从技术角度看，如果指针指向某个结构，可以首先用*对指针解除引用，然后使用普通的语法访问结构中的字段，如下面的代码所示，在此假定存在一个名为 `getEmployee()` 的函数。

```
Employee* anEmployee = getEmployee();
cout << (*anEmployee).salary << endl;
```

此语法有一点复杂。->(箭头)运算符允许同时对指针解除引用并访问字段。下面的代码与前面的代码等效，但阅读起来更方便：

```
Employee* anEmployee = getEmployee();
cout << anEmployee->salary << endl;
```

记住本章前面介绍的短路逻辑。这种做法可与指针一起使用，以免使用无效指针，如下所示：

```
bool isValidSalary = (anEmployee && anEmployee->salary > 0);
```

或者稍微详细一点：

```
bool isValidSalary = (anEmployee != nullptr && anEmployee->salary > 0);
```

仅当 `anEmployee` 有效时，才对其进行解引用以获取薪水。如果它是一个空指针，则逻辑运算短路，不再解引用 `anEmployee` 指针。

3. 动态分配的数组

堆也可以用于动态分配数组。使用 `new[]` 操作符可给数组分配内存：

```
int arraySize = 8;
int* myVariableSizedArray = new int[arraySize];
```

这条语句分配的内存用于存储 8 个整数，内存的大小与 `arraySize` 变量对应。图 1-3 显示了执行这条语句后堆栈和堆的情况。可以看到，指针变量仍在堆栈中，但动态创建的数组在堆中。

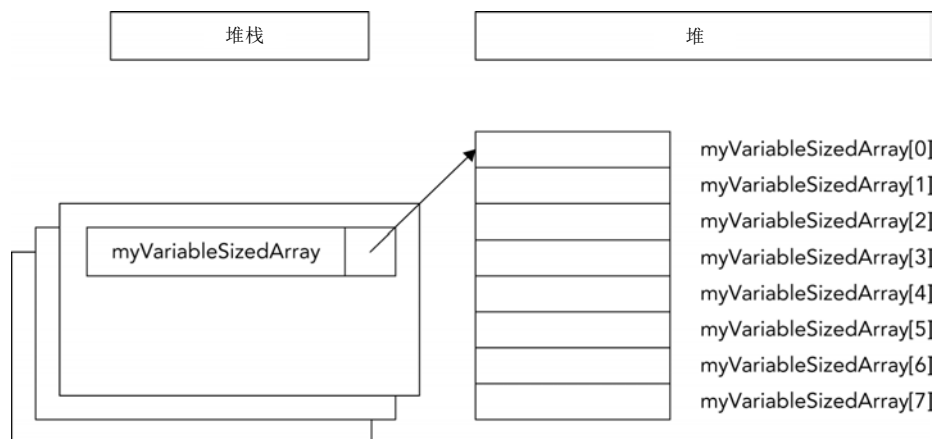


图 1-3

现在已经分配了内存，可将 `myVariableSizedArray` 当作基于堆栈的普通数组使用：

```
myVariableSizedArray[3] = 2;
```

使用完这个数组后，应该将其从堆中删除，这样其他变量就可以使用这块内存。在 C++ 中，可使用 `delete[]` 操作符完成这一任务：

```
delete[] myVariableSizedArray;
myVariableSizedArray = nullptr;
```

`delete` 后的方括号表明所删除的是一个数组！

注意：

避免使用 C 中的 `malloc()` 和 `free()`，而使用 `new` 和 `delete`，或者使用 `new[]` 和 `delete[]`。

警告：

在 C++ 中，每次调用 `new` 时，都必须相应地调用 `delete`；每次调用 `new[]` 时，都必须相应地调用 `delete[]`，以避免内存泄漏。如果未调用 `delete` 或 `delete[]`，或调用不匹配，会导致内存泄漏。第 7 章将讨论内存泄漏。

4. 空指针常量

在 C++11 之前，常量 `NULL` 用于表示空指针。将 `NULL` 定义为常量 0 会导致一些问题。分析下面的例子：

```
void func(char* str) {cout << "char* version" << endl;}
void func(int i) {cout << "int version" << endl;}

int main()
{
    func(NULL);
    return 0;
}
```

`main()` 函数通过参数 `NULL` 调用 `func()`，`NULL` 是一个空指针常量。换言之，该例要用空指针作为实参，调用 `func()` 的 `char*` 版本。但是，`NULL` 不是指针，而等价于整数 0，所以实际调用的是 `func()` 的整数版本。

可引入真正的空指针常量 `nullptr` 来解决这个问题。下面的代码调用了 `func()` 的 `char*` 版本：

```
func(nullptr);
```

5. 智能指针

为避免常见的内存问题，应使用智能指针替代通常的 C 样式“裸”指针。智能指针对象在超出作用域时，例如在函数执行完毕后，会自动释放内存。

C++ 中有两种最重要的智能指针：`std::unique_ptr` 和 `std::shared_ptr`。

`unique_ptr` 类似于普通指针，但在 `unique_ptr` 超出作用域或被删除时，会自动释放内存或资源。`unique_ptr` 只属于它指向的对象。`unique_ptr` 的一个优点是，内存和资源始终被释放，即使执行返回语句或抛出异常时（见稍后的讨论）。这简化了编码；例如，如果一个函数有多个返回语句，你不必记着在每个返回语句前释放资源。

要创建 `unique_ptr`，应当使用 `std::make_unique<>()`。例如，不要编写以下代码：

```
Employee* anEmployee = new Employee;
// ...
delete anEmployee;
```

而应当编写：

```
auto anEmployee = make_unique<Employee>();
```

注意这样一来，将不再调用 `delete`，因为这将自动完成。本章后面的“类型推断”小节将详细讨论 `auto` 关键字。这里只需要了解，`auto` 关键字告诉编译器自动推断变量的类型，因此你不必手动指定完整类型。

`unique_ptr` 是一个通用的智能指针，它可以指向任意类型的内存。所以它是一个模板。模板需要用尖括号指定模板参数。在尖括号中必须指定 `unique_ptr` 要指向的内存类型。模板详见第 12 章和第 22 章，而智能指针在本书开头介绍，以便在全书中使用。可以看出，它们很容易使用。

`make_unique()` 在 C++14 中引入。如果你的编译器与 C++14 不兼容，可使用如下形式的 `unique_ptr`（注意，现在必须将 `Employee` 类型指定两次）：

```
unique_ptr<Employee> anEmployee(new Employee);
```

可像普通指针那样使用 `anEmployee` 智能指针，例如：

```
if (anEmployee) {
    cout << "Salary: " << anEmployee->salary << endl;
}
```

`unique_ptr` 也可用于存储 C 风格的数组。下例创建一个包含 10 个 `Employee` 实例的数组，将其存储在 `unique_ptr` 中，并显示如何访问该数组中的元素：

```
auto employees = make_unique<Employee[]>(10);
cout << "Salary: " << employees[0].salary << endl;
```

`shared_ptr` 允许数据的分布式“所有权”。每次指定 `shared_ptr` 时，都递增一个引用计数，指出数据又多了一位“拥有者”。`shared_ptr` 超出作用域时，就递减引用计数。当引用计数为 0 时，就表示数据不再有任何拥有者，于是释放指针引用的对象。

要创建 `shared_ptr`，应当使用 `std::make_shared<>()`，它类似于 `make_unique<>()`：

```
auto anEmployee = make_shared<Employee>();
if (anEmployee) {
    cout << "Salary: " << anEmployee->salary << endl;
}
```

从 C++17 开始，也可将数组存储在 `shared_ptr` 中，而旧版 C++ 不允许这么做。但注意，此时不能使用 C++17 的 `make_shared<>()`。下面是一个示例：

```
shared_ptr<Employee[]> employees(new Employee[10]);
cout << "Salary: " << employees[0].salary << endl;
```

第 7 章将详细讨论内存管理和智能指针，但由于 `unique_ptr` 和 `shared_ptr` 的基本用法十分简单，它们总是用在本书的示例中。

注意：

普通的裸指针仅允许在不涉及所有权时使用，否则默认使用 `unique_ptr`。如果需要共享所有权，就使用 `shared_ptr`。如果知道 `auto_ptr`，应忘记它，因为 C++11/14 不赞成使用它，而 C++17 已经废弃了它。

1.2.3 const 的多种用法

在 C++ 中有多种方法使用 `const` 关键字。所有用法都是相关的，但存在微妙差别。第 11 章将介绍 `const` 的所有用法。下面将讲述最常见的用法。

1. 使用 const 定义常量

如果已经认为关键字 `const` 与常量有一定关系，就正确地揭示了它的一种用法。在 C 语言中，程序员经常使用预处理器的 `#define` 机制声明一个符号名称，其值在程序执行时不会变化，例如版本号。在 C++ 中，鼓励程序员使用 `const` 取代 `#define` 定义常量。使用 `const` 定义常量就像定义变量一样，只是编译器保证代码不会改变这个值。

```
const int versionNumberMajor = 2;
const int versionNumberMinor = 1;
const std::string productName = "Super Hyper Net Modulator";
```

2. 使用 const 保护参数

在 C++ 中，可将非 `const` 变量转换为 `const` 变量。为什么想这么做呢？这提供了一定程度的保护，防止其他代码修改变量。如果你调用同事编写的一个函数，并且想确保这个函数不会改变传递给它的参数值，可以告诉同事让函数采用 `const` 参数。如果这个函数试图改变参数的值，就不会编译。

在下面的代码中，调用 `mysteryFunction()` 时 `string*` 自动转换为 `const string*`。如果编写 `mysteryFunction()` 的人员试图修改所传递字符串的值，代码将无法编译。有绕过这个限制的方法，但是需要有意识地这么做，C++ 只是阻止无意识地修改 `const` 变量。

```
void mysteryFunction(const std::string* someString)
{
    *someString = "Test"; // Will not compile.
}

int main()
{
    std::string myString = "The string";
    mysteryFunction(&myString);
    return 0;
}
```

1.2.4 引用

C++ 中的引用允许给已有变量定义另一个名称，例如：

```
int x = 42;
int& xReference = x;
```

给类型附加一个 `&`，则指示相应的变量是引用。这不是一个普通变量，但仍在被使用；在幕后它实际上是一个指向原始变量的指针。变量 `x` 和引用变量 `xReference` 指向同一个值。如果通过其中一个更改值，则也可在另一个中看到更改。

1. 按引用传递

通常，给函数传递变量时，传递的是值。如果函数接收整型参数，实际上传入的是整数的一个副本，因此不会修改原始变量的值。C 中通常使用栈变量中的指针，以允许函数修改另一个堆栈帧中的变量。通过对指针进行解引用，函数可以更改表示该变量的内存(即使该变量不在当前堆栈帧中)。这种方法的问题在于，它将指针语法的复杂性带入了原本简单的任务中。

在 C++ 中，不是给函数传递指针，而是提供一种更好的机制，称为“按引用传递”，其中，参数是引用而非指针。下面是 `addOne()` 函数的两个版本。第一个版本不会影响传递给它的变量，因为变量是按照值传递的，因此函数接收的是传递给它的值的一个副本。第二个版本使用了引用，因此可以改变原始变量的值。

```
void addOne(int i)
{
    i++; // Has no real effect because this is a copy of the original
}

void addOne(int& i)
{
    i++; // Actually changes the original variable
}
```

调用具有整型引用参数的 `addOne()` 函数的语法与调用具有整型参数的 `addOne()` 函数没有区别。

```
int myInt = 7;
addOne(myInt);
```

注意：

两个 `addOne()` 函数的实现之间存在微妙区别。使用值传递的版本可以接收字面量而不会出现任何问题，例如 `addOne(3)` 是合法的。然而，如果向按引用传递的 `addOne()` 函数传递字面量，会导致编译错误。可使用下一节介绍的 `const` 引用解决该问题；也可使用右值引用解决这个问题，这一 C++ 高级特性将在第 9 章中讨论。

如果函数需要返回一个庞大的结构或类(见稍后的讨论)，则复制成本高昂；你经常看到，函数为这样的结构或类使用非 `const` 引用，此后进行修改，而非直接返回。很久以前，推荐使用这种方法，以免在从函数返回结构或类时由于创建副本而影响性能。从 C++11 开始，再也不必这么做了；原因在于有了 `move` 语义，`move` 语义直接从函数返回结构或类，不需要任何复制，十分有效。第 9 章将详细讨论 `move` 语义。

2. 按 `const` 引用传递

经常可以看到，代码为函数使用 `const` 引用参数。乍看上去这有点自相矛盾，引用参数允许在另一种环境中改变变量的值，而 `const` 应该会阻止这种改变。

`const` 引用参数的主要价值在于效率。当向函数传递值时，会制作一个完整副本。当传递引用时，实际上只是传递一个指向原始数据的指针，这样计算机就不需要制作副本。通过传递 `const` 引用，可做到二者兼顾——不需要副本，原始变量也不会修改。

在处理对象时，`const` 引用会变得更重要，因为对象可能比较庞大，复制对象可能需要很大的代价。第 11 章将讲述如何处理此类复杂问题。下面的示例说明了如何把 `std::string` 作为 `const` 引用传递给函数：

```
void printString(const std::string& myString)
{
    std::cout << myString << std::endl;
}

int main()
{
    std::string someString = "Hello World";
    printString(someString);
    printString("Hello World"); // Passing literals works
    return 0;
}
```

注意：

如果需要给函数传递对象，最好按 `const` 引用（而非值）传递它。这样可以防止多余的复制。如果函数需要修改对象，则为其传递非 `const` 引用。

1.2.5 异常

C++ 是一种非常灵活的语言，但并不是非常安全。编译器允许编写改变随机内存地址或者尝试除以 0 的代

码(计算机无法处理无穷大的数值)。异常就是试图增加一点安全性的语言特性。

异常是一种无法预料的情形。例如,如果编写一个获取 Web 页面的函数,就有几件事情可能出错,包含页面的 Internet 主机可能被关闭,页面可能是空白的,或者连接可能会丢失。处理这种情况的一种方法是,从函数返回特定的值,如 `nullptr` 或其他错误代码。异常提供了处理此类问题的更好方法。

异常伴随着一些新术语。当某段代码检测到异常时,就会抛出一个异常。另一段代码会捕获这个异常并执行恰当的操作。下例给出一个名为 `divideNumbers()` 的函数,如果调用者传递给分母的值 0,就会抛出一个异常。使用 `std::invalid_argument` 时需要 `<stdexcept>`。

```
double divideNumbers(double numerator, double denominator)
{
    if (denominator == 0) {
        throw invalid_argument("Denominator cannot be 0.");
    }
    return numerator / denominator;
}
```

当执行 `throw` 行时,函数将立刻结束而不会返回值。如果调用者将函数调用放到 `try/catch` 块中,就可以捕获异常并进行处理,如下所示:

```
try {
    cout << divideNumbers(2.5, 0.5) << endl;
    cout << divideNumbers(2.3, 0) << endl;
    cout << divideNumbers(4.5, 2.5) << endl;
} catch (const invalid_argument& exception) {
    cout << "Exception caught: " << exception.what() << endl;
}
```

第一次调用 `divideNumbers()` 成功执行,结果会输出给用户。第二次调用会抛出一个异常,不会返回值,唯一的输出是捕获异常时输出的错误信息。第三次调用根本不会执行,因为第二次调用抛出了一个异常,导致程序跳转到 `catch` 块。前面代码块的输出是:

```
5
An exception was caught: Denominator cannot be 0.
```

C++ 的异常非常灵活,为正确使用异常,需要理解抛出异常时堆栈变量的行为,必须正确捕获并处理必要的异常。前面的示例中使用了内建的 `std::invalid_argument` 类型,但最好根据所抛出的具体错误,编写自己的异常类型。最后, C++ 编译器并不强制要求捕获可能发生的所有异常。如果代码从不捕获任何异常,但有异常抛出,程序自身会捕获异常并终止。第 14 章将进一步讨论异常的这些更复杂的方面。

1.2.6 类型推断

类型推断允许编译器自动推断出表达式的类型。类型推断有两个关键字: `auto` 和 `decltype`。

1. 关键字 auto

关键字 `auto` 有多种完全不同的含义:

- 推断函数的返回类型,如前所述。
- 结构化绑定,如前所述。
- 推断表达式的类型,如前所述。
- 推断非类型模板参数的类型,见第 12 章。
- `decltype(auto)`,见第 12 章。
- 其他函数语法,见第 12 章。
- 通用 `lambda` 表达式,见第 18 章。

`auto` 可用于告诉编译器,在编译时自动推断变量的类型。下面的代码演示了在这种情况下,关键字 `auto` 最简单的用法:

```
auto x = 123;    // x will be of type int
```

在这个示例中,输入 `auto` 和输入 `int` 的效果没什么区别,但 `auto` 对较复杂的类型会更有用。假定 `getFoo()`

函数有一个复杂的返回类型。如果希望把调用该函数的结果赋予一个变量，就可以输入该复杂类型，也可以简单地使用 `auto`，让编译器推断出该类型：

```
auto result = getFoo();
```

这样，你可方便地更改函数的返回类型，而不需要更新代码中调用该函数的所有位置。

但使用 `auto` 去除了引用和 `const` 限定符。假设有以下函数：

```
#include <string>

const std::string message = "Test";

const std::string& foo()
{
    return message;
}
```

可以调用 `foo()`，把结果存储在一个变量中，将该变量的类型指定为 `auto`，如下所示：

```
auto f1 = foo();
```

因为 `auto` 去除了引用和 `const` 限定符，且 `f1` 是 `string` 类型，所以建立一个副本。如果希望 `f1` 是一个 `const` 引用，就可以明确将它建立一个引用，并标记为 `const`，如下所示：

```
const auto& f2 = foo();
```

警告：

始终要记住，`auto` 去除了引用和 `const` 限定符，从而会创建副本！如果不需要副本，可使用 `auto&` 或 `const auto&`。

2. 关键字 decltype

关键字 `decltype` 把表达式作为实参，计算出该表达式的类型。例如：

```
int x = 123;
decltype(x) y = 456;
```

在这个示例中，编译器推断出 `y` 的类型是 `int`，因为这是 `x` 的类型。

`auto` 与 `decltype` 的区别在于，`decltype` 未去除引用和 `const` 限定符。再来分析返回 `const string` 引用的 `foo()` 函数。按如下方式使用 `decltype` 定义 `f2`，导致 `f2` 的类型为 `const string&`，从而不生成副本：

```
decltype(foo()) f2 = foo();
```

刚开始不会觉得 `decltype` 有多大价值。但在模板环境中，`decltype` 会变得十分强大，详见第 12 和 22 章。

1.3 作为面向对象语言的 C++

如果你是一位 C 程序员，可能会认为本章讲述的内容到目前为止只是传统 C 语言的补充。顾名思义，C++ 语言在很多方面只是“更好的 C”。这种观点忽略了一个重点：与 C 不同，C++ 是一种面向对象的语言。

面向对象程序设计(OOP)是一种完全不同的、更趋自然的编码方式。如果习惯使用过程语言，如 C 或者 Pascal，不要担心。第 5 章讲述将观念转换到面向对象范型所需要的所有背景知识。如果你已经了解 OOP 的理论，下面的内容将帮助你加速了解(或者回顾)基本的 C++ 对象语法。

1.3.1 定义类

类定义了对象的特征。在 C++ 中，类通常在头文件(.h)中声明，在对应的源文件(.cpp)中定义其非内联方法和静态数据成员。

下面的示例定义了一个基本的机票类。这个类可根据飞行的里程数以及顾客是不是“精英超级奖励计划”的成员计算票价。这个定义首先声明一个类名，在大括号内声明了类的数据成员(属性)以及方法(行为)。每个数据成员以及方法都具有特定的访问级别：`public`、`protected` 或 `private`。这些标记可按任意顺序出现，也可重复使

用。**public** 成员可在类的外部访问，**private** 成员不能在类的外部访问，推荐把所有的数据成员都声明为 **private**，在需要时，可通过 **public** 读取器和设置器来访问它们。这样，就很容易改变数据的表达方式，同时使 **public** 接口保持不变。关于 **protected** 的用法，将在第 5 和 10 章中介绍“继承”时讲解。

```
#include <string>

class AirlineTicket
{
public:
    AirlineTicket();
    ~AirlineTicket();

    double calculatePriceInDollars() const;

    const std::string& getPassengerName() const;
    void setPassengerName(const std::string& name);

    int getNumberOfMiles() const;
    void setNumberOfMiles(int miles);

    bool hasEliteSuperRewardsStatus() const;
    void setHasEliteSuperRewardsStatus(bool status);
private:
    std::string mPassengerName;
    int mNumberOfMiles;
    bool mHasEliteSuperRewardsStatus;
};
```

本书遵循这样一个约定：在类的每个数据成员之前加上小写字母 **m**，如 **mPassengerName**。

注意：

为遵循 **const** 正确性原则，最好将不改变对象的任何数据成员的成员函数声明为 **const**。相对于非 **const** 成员函数“修改器”，这些成员函数也称为“检测器”。

与类同名但没有返回类型的方法是构造函数，当创建类的对象时会自动调用构造函数。~之后紧接着类名的方法是析构函数，当销毁对象时会自动调用。

使用构造函数，可通过两种方法来初始化数据成员。推荐的做法是使用构造函数初始化器(constructor initializer)，即在构造函数名称之后加上冒号。下面是包含构造函数初始化器的 **AirlineTicket** 构造函数：

```
AirlineTicket::AirlineTicket()
: mPassengerName("Unknown Passenger")
, mNumberOfMiles(0)
, mHasEliteSuperRewardsStatus(false)
{
}
```

第二种方法是将初始化任务放在构造函数体中，如下所示：

```
AirlineTicket::AirlineTicket()
{
    // Initialize data members
    mPassengerName = "Unknown Passenger";
    mNumberOfMiles = 0;
    mHasEliteSuperRewardsStatus = false;
}
```

如果构造函数只是初始化数据成员，而不做其他事情，实际上就没必要使用构造函数，因为可在类定义中直接初始化数据成员。例如，不编写 **AirlineTicket** 构造函数，而是修改类定义中数据成员的定义，如下所示：

```
private:
    std::string mPassengerName = "Unknown Passenger";
    int mNumberOfMiles = 0;
    bool mHasEliteSuperRewardsStatus = false;
```

如果类还需要执行其他一些初始化类型，如打开文件、分配内存等，则需要编写构造函数进行处理。

下面是 **AirlineTicket** 类的析构函数：

```
AirlineTicket::~AirlineTicket()
{
}
```

```
    // Nothing much to do in terms of cleanup
}
```

这个析构函数什么都不做，因此可从类中删除。这里之所以显示它，是为了让你了解析构函数的语法。如果需要执行一些清理，如关闭文件、释放内存等，则需要使用析构函数。第 8 和 9 章详细讨论析构函数。

一些 `AirlineTicket` 类方法的定义如下所示：

```
double AirlineTicket::calculatePriceInDollars() const
{
    if (hasEliteSuperRewardsStatus()) {
        // Elite Super Rewards customers fly for free!
        return 0;
    }
    // The cost of the ticket is the number of miles times 0.1.
    // Real airlines probably have a more complicated formula!
    return getNumberOfMiles() * 0.1;
}

const string& AirlineTicket::getPassengerName() const
{
    return mPassengerName;
}

void AirlineTicket::setPassengerName(const string& name)
{
    mPassengerName = name;
}

// Other get and set methods omitted for brevity.
```

1.3.2 使用类

下面的示例程序使用了 `AirlineTicket` 类。这个示例创建的两个 `AirlineTicket` 对象分别基于堆栈和堆。

```
AirlineTicket myTicket; // Stack-based AirlineTicket
myTicket.setPassengerName("Sherman T. Socketwrench");
myTicket.setNumberOfMiles(700);
double cost = myTicket.calculatePriceInDollars();
cout << "This ticket will cost $" << cost << endl;

// Heap-based AirlineTicket with smart pointer
auto myTicket2 = make_unique<AirlineTicket>();
myTicket2->setPassengerName("Laudimore M. Hallidue");
myTicket2->setNumberOfMiles(2000);
myTicket2->setHasEliteSuperRewardsStatus(true);
double cost2 = myTicket2->calculatePriceInDollars();
cout << "This other ticket will cost $" << cost2 << endl;
// No need to delete myTicket2, happens automatically

// Heap-based AirlineTicket without smart pointer (not recommended)
AirlineTicket* myTicket3 = new AirlineTicket();
// ... Use ticket 3
delete myTicket3; // delete the heap object!
```

上面的示例代码显示了创建和使用类的一般语法。当然，还有许多内容需要学习。第 8~10 章将更深入地讲述 C++ 定义类的特定机制。

1.4 统一初始化

在 C++11 之前，初始化类型并非总是统一的。例如，考虑下面的两个定义，其中一个作为结构，另一个作为类：

```
struct CircleStruct
{
    int x, y;
    double radius;
};

class CircleClass
```

```

{
    public:
        CircleClass(int x, int y, double radius)
            : mX(x), mY(y), mRadius(radius) {}
    private:
        int mX, mY;
        double mRadius;
};

```

在 C++11 之前，`CircleStruct` 类型变量和 `CircleClass` 类型变量的初始化是不同的：

```

CircleStruct myCircle1 = {10, 10, 2.5};
CircleClass myCircle2(10, 10, 2.5);

```

对于结构版本，可使用 `{...}` 语法。然而，对于类版本，需要使用函数符号 `(...)` 调用构造函数。

自 C++11 以后，允许一律使用 `{...}` 语法初始化类型，如下所示：

```

CircleStruct myCircle3 = {10, 10, 2.5};
CircleClass myCircle4 = {10, 10, 2.5};

```

定义 `myCircle4` 时将自动调用 `CircleClass` 的构造函数。甚至等号也是可选的，因此下面的代码与前面的代码等价：

```

CircleStruct myCircle5{10, 10, 2.5};
CircleClass myCircle6{10, 10, 2.5};

```

统一初始化并不局限于结构和类，它还可用于初始化 C++ 中的任何内容。例如，下面的代码把所有 4 个变量都初始化为 3：

```

int a = 3;
int b(3);
int c = {3}; // Uniform initialization
int d{3};    // Uniform initialization

```

统一初始化还可用于将变量初始化为 0；使用默认构造函数构造对象，将基本整数类型（如 `char` 和 `int` 等）初始化为 0，将浮点类型初始化为 0.0，将指针类型初始化为 `nullptr`。为此，只需要指定一系列空的大括号，例如：

```

int e{}; // Uniform initialization, e will be 0

```

使用统一初始化还可以阻止窄化(narrowing)。C++ 隐式地执行窄化，例如：

```

void func(int i) { /* ... */ }

int main()
{
    int x = 3.14;
    func(3.14);
}

```

这两种情况下，C++ 在对 `x` 赋值或调用 `func()` 之前，会自动将 3.14 截断为 3。注意有些编译器会针对窄化给出警告信息，而另一些编译器则不会。使用统一初始化，如果编译器完全支持 C++11 标准，`x` 的赋值和 `func()` 的调用都会生成编译错误：

```

void func(int i) { /* ... */ }

int main()
{
    int x = {3.14}; // Error because narrowing
    func({3.14});  // Error because narrowing
}

```

统一初始化还可用来初始化动态分配的数组：

```

int* pArray = new int[4]{0, 1, 2, 3};

```

统一初始化还可在构造函数初始化器中初始化类成员数组：

```

class MyClass
{
    public:

```

```

        MyClass() : mArray{0, 1, 2, 3} {}
    private:
        int mArray[4];
};

```

统一初始化还可用于标准库容器，如 `std::vector`，见稍后的描述。

直接列表初始化与复制列表初始化

有两种初始化类型使用包含在大括号中的初始化列表：

- 复制列表初始化：`T obj = {arg1, arg2, ...};`
- 直接列表初始化：`T obj {arg1, arg2, ...};`

在 C++17 中，与 `auto` 类型推断相结合，直接列表初始化与复制列表初始化存在重要区别。

从 C++17 开始，可得到以下结果：

```

// Copy list initialization
auto a = {11};           // initializer_list<int>
auto b = {11, 22};       // initializer_list<int>

// Direct list initialization
auto c {11};             // int
auto d {11, 22};         // Error, too many elements.

```

注意，对于复制列表初始化，放在大括号中的初始化器的所有元素都必须使用相同的类型。例如，以下代码无法编译：

```
auto b = {11, 22.33}; // Compilation error
```

在早期标准版本(C++11/14)中，复制列表初始化和直接列表初始化会推导出 `initializer_list<>`：

```

// Copy list initialization
auto a = {11};           // initializer_list<int>
auto b = {11, 22};       // initializer_list<int>

// Direct list initialization
auto c {11};             // initializer_list<int>
auto d {11, 22};         // initializer_list<int>

```

1.5 标准库

C++ 具有标准库，其中包含许多有用的类，在代码中可方便地使用这些类。使用标准库中类的好处是不需要重新创建某些类，也不需要浪费时间去实现系统已经自动实现的内容。另一好处是标准库中的类已经经过成千上万用户的严格测试和验证。标准库中类的性能也比较高，因此使用这些类比使用自己的类效率更高。

标准库中可用的功能非常多。第 16~20 章将详细讲述标准库。当开始使用 C++ 时，最好立刻了解标准库可以做什么。如果你是一位 C 程序员，这一点尤其重要。作为 C 程序员，你使用 C++ 时可能会以 C 的方式解决问题。然而使用 C++ 的标准库类可以更方便、安全地解决问题。

本章前面已经介绍了标准库中的一些类，例如 `std::string`、`std::array`、`std::vector`、`std::unique_ptr` 和 `std::shared_ptr`。第 16~20 章将介绍更多的类。

1.6 第一个有用的 C++ 程序

下面的程序建立一个雇员数据库，在前面讨论结构时曾将其用作示例。在此，将使用本章前面讲述的许多特性来完成一个功能完整的 C++ 程序。这个实际的示例使用了类、异常、流、`vector`、名称空间、引用以及其他语言特性。

1.6.1 雇员记录系统

管理公司雇员记录的程序应该灵活并具有有效的功能。这个程序包含的功能有：

- 添加雇员
- 解雇雇员
- 雇员晋升
- 查看所有雇员，包括过去以及现在的雇员
- 查看所有当前雇员
- 查看所有以前雇员

程序的代码分为三个部分：**Employee** 类封装了单个雇员的信息，**Database** 类管理公司的所有雇员，单独的用户界面提供程序的接口。

1.6.2 Employee 类

Employee 类维护某个雇员的全部信息，该类的方法提供了查询以及修改信息的途径。**Employee** 还知道如何在控制台显示自身。此外还存在调整雇员薪水和就业状态的方法。

1. Employee.h

Employee.h 文件声明了 **Employee** 类，下面将单独讨论这个文件的各个部分。

第一行包括 `#pragma once`，以防止文件被包含多次，此外还包含 `string` 功能。

代码还声明后面的代码(包含在大括号中)将位于 **Records** 名称空间。为使用特定代码，整个程序都会用到 **Records** 名称空间。

```
#pragma once
#include <string>
namespace Records {
```

下面的常量代表新雇员的默认起薪，位于 **Records** 名称空间。**Records** 名称空间中的其他代码可以将这个常量作为 `kDefaultStartingSalary` 访问。在其他位置，必须通过 `Records::kDefaultStartingSalary` 来引用它。

```
const int kDefaultStartingSalary = 30000;
```

注意，本书使用以下约定：给常量加前缀 `k`(小写字母)。这源于德语单词 **Konstant**，意思是“顾问”。

在此声明了 **Employee** 类及其公有方法。`promote()` 以及 `demote()` 方法的整型参数都具有指定的默认值。通过这种方法，其他代码可以忽略整型参数，将自动使用默认值。

在此，许多获取器和设置器提供了修改雇员信息或查询雇员当前信息的机制。

Employee 类包含一个显式的默认构造函数，如第 8 章所述。它还包含接收姓名的构造函数。

```
class Employee
{
public:
    Employee() = default;
    Employee(const std::string& firstName,
             const std::string& lastName);

    void promote(int raiseAmount = 1000);
    void demote(int demeritAmount = 1000);
    void hire(); // Hires or rehires the employee
    void fire(); // Dismisses the employee
    void display() const; // Outputs employee info to console

    // Getters and setters
    void setFirstName(const std::string& firstName);
    const std::string& getFirstName() const;

    void setLastName(const std::string& lastName);
    const std::string& getLastName() const;

    void setEmployeeNumber(int employeeNumber);
```

```

int getEmployeeNumber() const;

void setSalary(int newSalary);
int getSalary() const;

bool isHired() const;

```

最后将数据成员声明为 **private**，这样其他部分的代码将无法直接修改它们。获取器和设置器提供了修改或查询这些值的唯一公有途径。数据成员也在这里(而非构造函数中)进行初始化。默认情况下，新雇员无姓名，雇员编号为 -1；另外，会指定默认起薪以及受雇状态。

```

private:
    std::string mFirstName;
    std::string mLastName;
    int mEmployeeNumber = -1;
    int mSalary = kDefaultStartingSalary;
    bool mHired = false;
};
}

```

2. Employee.cpp

构造函数接收姓和名，只设置相应的数据成员：

```

#include <iostream>
#include "Employee.h"

using namespace std;

namespace Records {
    Employee::Employee(const std::string& firstName,
                       const std::string& lastName)
        : mFirstName(firstName), mLastName(lastName)
    {
    }
}

```

promote()和 **demote()**方法只是用一些新值调用 **setSalary()**方法。注意，整型参数的默认值不显示在源文件中；它们只能出现在函数声明中，不能出现在函数定义中。

```

void Employee::promote(int raiseAmount)
{
    setSalary(getSalary() + raiseAmount);
}

void Employee::demote(int demeritAmount)
{
    setSalary(getSalary() - demeritAmount);
}

```

hire()和 **fire()**方法正确设置了 **mHired** 数据成员。

```

void Employee::hire()
{
    mHired = true;
}

void Employee::fire()
{
    mHired = false;
}

```

display()方法使用控制台输出流显示当前雇员的信息。由于这段代码是 **Employee** 类的一部分，因此可直接访问数据成员(如 **mSalary**)，而不需要使用 **getSalary()**获取器。然而，使用获取器和设置器(当存在时)是一种好的风格，甚至在类的内部也是如此。

```

void Employee::display() const
{
    cout << "Employee: " << getLastName() << ", " << getFirstName() << endl;
    cout << "-----" << endl;
    cout << (isHired() ? "Current Employee" : "Former Employee") << endl;
    cout << "Employee Number: " << getEmployeeNumber() << endl;
    cout << "Salary: $" << getSalary() << endl;
    cout << endl;
}

```

许多获取器和设置器执行获取值以及设置值的任务。即使这些方法看起来微不足道，但是使用这些微不足道的获取器和设置器，仍然优于将数据成员设置为 `public`。将来，你可能想在 `setSalary()` 方法中执行边界检查，获取器和设置器也能简化调试，因为可在其中设置断点，在检索或设置值时检查它们。另一个原因是决定修改类中存储数据的方式时，只需要修改这些获取器和设置器。

```
// Getters and setters
void Employee::setFirstName(const string& firstName)
{
    mFirstName = firstName;
}

const string& Employee::getFirstName() const
{
    return mFirstName;
}
// ... other getters and setters omitted for brevity
}
```

3. EmployeeTest.cpp

当编写一个类时，最好对其进行独立测试。以下代码在 `main()` 函数中针对 `Employee` 类执行了一些简单操作。当确信 `Employee` 类可正常运行后，应该删除这个文件，或将这个文件注释掉，这样就不会编译具有多个 `main()` 函数的代码。

```
#include <iostream>
#include "Employee.h"

using namespace std;
using namespace Records;

int main()
{
    cout << "Testing the Employee class." << endl;
    Employee emp;
    emp.setFirstName("John");
    emp.setLastName("Doe");
    emp.setEmployeeNumber(71);
    emp.setSalary(50000);
    emp.promote();
    emp.promote(50);
    emp.hire();
    emp.display();
    return 0;
}
```

另一种测试各个类的方法是使用单元测试，详见第 26 章中的讨论。

1.6.3 Database 类

`Database` 类使用标准库中的 `std::vector` 类来存储 `Employee` 对象。

1. Database.h

由于数据库会自动给新雇员指定一个雇员号，因此定义一个常量作为编号的开始：

```
#pragma once
#include <iostream>
#include <vector>
#include "Employee.h"

namespace Records {
    const int kFirstEmployeeNumber = 1000;
```

数据库可根据提供的姓名方便地添加一个新雇员。为方便起见，这个方法返回一个新雇员的引用。外部代码也可通过调用 `getEmployee()` 方法来获得雇员的引用。为这个方法声明了两个版本，一个允许按雇员号进行检索，另一个要求提供雇员的姓名。

```
class Database
```

```

{
    public:
        Employee& addEmployee(const std::string& firstName,
                               const std::string& lastName);
        Employee& getEmployee(int employeeNumber);
        Employee& getEmployee(const std::string& firstName,
                               const std::string& lastName);

```

由于数据库是所有雇员记录的中心存储库，因此具有输出所有雇员、当前在职雇员以及已离职雇员的方法。

```

    void displayAll() const;
    void displayCurrent() const;
    void displayFormer() const;

```

mEmployees 包含 Employee 对象。数据成员 mNextEmployeeNumber 跟踪新雇员的雇员号，使用 kFirstEmployeeNumber 常量进行初始化。

```

    private:
        std::vector<Employee> mEmployees;
        int mNextEmployeeNumber = kFirstEmployeeNumber;
};

```

2. Database.cpp

addEmployee()方法创建一个新的 Employee 对象，在其中填充信息并将其添加到 vector 中。注意当使用了这个方法后，数据成员 mNextEmployeeNumber 的值会递增，因此下一个雇员将获得新编号。

```

#include <iostream>
#include <stdexcept>
#include "Database.h"

using namespace std;

namespace Records {
    Employee& Database::addEmployee(const string& firstName,
                                     const string& lastName)
    {
        Employee theEmployee(firstName, lastName);
        theEmployee.setEmployeeNumber(mNextEmployeeNumber++);
        theEmployee.hire();
        mEmployees.push_back(theEmployee);
        return mEmployees[mEmployees.size() - 1];
    }
}

```

由于两个版本的 getEmployee()运行方式类似，因此在此只显示了其中一个版本。该方法使用基于区间的 for 循环，遍历 mEmployees 中的所有雇员，并检查是否有与传递给这个方法的信息匹配的雇员。如果没有发现匹配的信息，就会抛出异常。

```

Employee& Database::getEmployee(int employeeNumber)
{
    for (auto& employee : mEmployees) {
        if (employee.getEmployeeNumber() == employeeNumber) {
            return employee;
        }
    }
    throw logic_error("No employee found.");
}

```

所有显示方法都采用相似的算法。这些方法遍历所有雇员，如果符合显示标准，就通知雇员将自身显示到控制台中。displayFormer()类似于 displayCurrent()。

```

void Database::displayAll() const
{
    for (const auto& employee : mEmployees) {
        employee.display();
    }
}

void Database::displayCurrent() const
{
    for (const auto& employee : mEmployees) {
        if (employee.isHired())
            employee.display();
    }
}

```



```

    }
}

```

3. DatabaseTest.cpp

用于数据库基本功能的简单测试如下所示：

```

#include <iostream>
#include "Database.h"

using namespace std;
using namespace Records;

int main()
{
    Database myDB;
    Employee& emp1 = myDB.addEmployee("Greg", "Wallis");
    emp1.fire();

    Employee& emp2 = myDB.addEmployee("Marc", "White");
    emp2.setSalary(100000);

    Employee& emp3 = myDB.addEmployee("John", "Doe");
    emp3.setSalary(10000);
    emp3.promote();

    cout << "all employees: " << endl << endl;
    myDB.displayAll();

    cout << endl << "current employees: " << endl << endl;
    myDB.displayCurrent();

    cout << endl << "former employees: " << endl << endl;
    myDB.displayFormer();
}

```

1.6.4 用户界面

程序的最后一部分是基于菜单的用户界面，可让用户方便地使用雇员数据库。

`main()` 函数是一个显示菜单的循环，执行被选中的操作，然后重新开始循环。对于大多数的操作都定义了独立的函数。对于显示雇员之类的简单操作，则将实际代码放在对应的情况(case)中。

```

#include <iostream>
#include <stdexcept>
#include <exception>
#include "Database.h"

using namespace std;
using namespace Records;

int displayMenu();
void doHire(Database& db);
void doFire(Database& db);
void doPromote(Database& db);
void doDemote(Database& db);

int main()
{
    Database employeeDB;
    bool done = false;
    while (!done) {
        int selection = displayMenu();
        switch (selection) {
            case 0:
                done = true;
                break;
            case 1:
                doHire(employeeDB);
                break;
            case 2:
                doFire(employeeDB);
                break;

```

```

        case 3:
            doPromote(employeeDB);
            break;
        case 4:
            employeeDB.displayAll();
            break;
        case 5:
            employeeDB.displayCurrent();
            break;
        case 6:
            employeeDB.displayFormer();
            break;
        default:
            cerr << "Unknown command." << endl;
            break;
    }
}
return 0;
}

```

`displayMenu()`函数输出菜单并获取用户输入。在此假定用户能够“正确地输入”，当需要一个数字时就会输入一个数字，这一点很重要。在阅读了第 13 章有关 I/O 的内容后，你就会知道如何防止输入错误信息。

```

int displayMenu()
{
    int selection;
    cout << endl;
    cout << "Employee Database" << endl;
    cout << "-----" << endl;
    cout << "1) Hire a new employee" << endl;
    cout << "2) Fire an employee" << endl;
    cout << "3) Promote an employee" << endl;
    cout << "4) List all employees" << endl;
    cout << "5) List all current employees" << endl;
    cout << "6) List all former employees" << endl;
    cout << "0) Quit" << endl;
    cout << endl;
    cout << "---> ";
    cin >> selection;
    return selection;
}

```

`doHire()`函数获取用户输入的新的雇员姓名，并通知数据库添加这个雇员。

```

void doHire(Database& db)
{
    string firstName;
    string lastName;

    cout << "First name? ";
    cin >> firstName;

    cout << "Last name? ";
    cin >> lastName;

    db.addEmployee(firstName, lastName);
}

```

`doFire()`以及 `doPromote()`都会要求数据库根据雇员号找到雇员的记录，然后使用 `Employee` 对象的 `public` 方法进行修改。

```

void doFire(Database& db)
{
    int employeeNumber;

    cout << "Employee number? ";
    cin >> employeeNumber;

    try {
        Employee& emp = db.getEmployee(employeeNumber);
        emp.fire();
        cout << "Employee " << employeeNumber << " terminated." << endl;
    } catch (const std::logic_error& exception) {
        cerr << "Unable to terminate employee: " << exception.what() << endl;
    }
}

```

```
void doPromote(Database& db)
{
    int employeeNumber;
    int raiseAmount;

    cout << "Employee number? ";
    cin >> employeeNumber;

    cout << "How much of a raise? ";
    cin >> raiseAmount;

    try {
        Employee& emp = db.getEmployee(employeeNumber);
        emp.promote(raiseAmount);
    } catch (const std::logic_error& exception) {
        cerr << "Unable to promote employee: " << exception.what() << endl;
    }
}
```

1.6.5 评估程序

前面的程序涵盖了许多主题，从最简单的到较复杂的都有。可采用多种方法扩展这个程序。例如，用户界面(UI)没有公开 `Database` 或 `Employee` 类的全部功能。可修改 UI，以包含这些特性。还可修改 `Database` 类，以从 `mEmployees` 中删除被解雇的雇员。

如果不理解程序的某些部分，参考前面的内容以回顾这些主题。如果仍不甚明了，最好的学习方法是编写代码并查看结果。例如，如果不确定如何使用条件运算符，可编写一个简单的 `main()` 函数进行测试。

1.7 本章小结

现在你已经了解了 C++ 的基本知识，为成为专业 C++ 程序员做好了准备。在开始深入学习本书后面的 C++ 语言知识时，可查阅本章以回顾需要复习的内容。为了回顾那些被遗忘的概念，只需要查看本章的一些示例代码。

你编写的每个程序都必须以这样或那样的方式使用字符串。为此，下一章将深入讲解如何在 C++ 中处理字符串。

第 2 章

使用 `string` 和 `string_view`

本章内容

- C 风格字符串和 C++ 字符串的区别
- C++ `std::string` 类的细节
- 使用 `std::string_view` 的原因
- 原始字符串字面量

从 wrox.com 下载本章的示例代码

注意，可访问本书网站 www.wrox.com/go/proc++4e，从 **Download Code** 选项卡下载本章的所有示例代码。

你编写的每个应用程序都会使用某种类型的字符串。使用老式 C 语言时，没有太多选择，只能使用普通的以 `null` 结尾的字符数组来表示字符串。遗憾的是，这种表示方式会导致很多问题，例如会导致安全攻击的缓冲区溢出。C++ 标准库包含了一个安全易用的 `std::string` 类，这个类没有这些缺点。

字符串十分重要，本章将详细讨论字符串。

2.1 动态字符串

在将字符串当成一等对象支持的语言中，字符串有很多有吸引人的特性，例如可扩展至任意大小，或能提取或替换子字符串。在其他语言(如 C 语言)中，字符串几乎就像后加入的功能；C 语言中并没有真正好用的 `string` 数据类型，只有固定的字节数组。“字符串库”只不过是一组非常原始的函数，甚至没有边界检查的功能。C++ 提供了 `string` 类型作为一等数据类型。

2.1.1 C 风格的字符串

在 C 语言中，字符串表示为字符的数组。字符串中的最后一个字符是 `null` 字符(`'\0'`)，这样，操作字符串的代码就知道字符串在哪里结束。官方将这个 `null` 字符定义为 `NUL`，这个拼写中只有一个 L，而不是两个 L。`NUL` 和 `NULL` 指针是两回事。尽管 C++ 提供了更好的字符串抽象，但理解 C 语言中使用的字符串技术非常重要，因为在 C++ 程序设计中仍可能使用这些技术。最常见的一种情况是 C++ 程序调用某个第三方库中(作为操作系统接口的一部分)用 C 语言编写的接口。